Package 'metronautica'

October 22, 2026

Type Package

Title Wearable Triaxial Accelerometer Metrology and Signal Processing

Version 0.1.0

Date 2026-10-22

Language en-GB

NeedsCompilation no

Author Millen James Theophilus [aut, cre]
(<https://orcid.org/0000-0001-9823-7177>)

Maintainer Millen James Theophilus <m.j.theophilus@lboro.ac.uk>

Description Native wearable triaxial accelerometer parsing, signal diagnostics, clipping, static and dynamic characterisation, filtering and reporting. Metrological terminology follows the International Vocabulary of Metrology, JCGM (2012) <doi:10.59161/JCGM200-2012>. Standalone activity counts implement the published ActiGraph algorithm of Neishabouri et al. (2022) <doi:10.1038/s41598-022-16003-x>, with its published filter as the default and configurable band-pass cutoffs for modified counts. Developed for the wearable-accelerometer metrology research article under submission for publication, with accompanying analyses and data in WAM <https://github.com/miltheo/wam>.

License GPL-3

URL https://github.com/miltheo/metronautica

BugReports https://github.com/miltheo/metronautica/issues

Encoding UTF-8

Depends R (>= 4.1.0)

Imports
tibble, dplyr (>= 1.0.0), data.table, digest, jsonlite, signal, ggplot2 (>= 3.4.0), ggrepel (>= 0.9.1), scales, stats, utils, grid, tools

Suggests plotly, GENEAread, GGIRread, read.gt3x, MIMSunit, TSEntropies

Collate 'm_qc.R'
'm_parsers.R'
'm_accel_clipper.R'
'm_static.R'
'm_dynamic.R'
'm_filter.R'

'm_accel_summary_suite.R'
'm_agcounts.R'
'm_plotter.R'
'm_sample_data.R'

Contents

metronautica-package . . . 3
accel_meta . . . 4
accel_metric_agcounts . . . 5
accel_metric_ai . . . 8
accel_metric_enmo . . . 9
accel_metric_mad . . . 10
accel_metric_mims . . . 11
accel_metric_odba . . . 13
accel_metric_rocam . . . 15
accel_metric_svmbp . . . 16
accel_metric_svmg . . . 17
accel_metric_vm . . . 18
accel_metric_z_angle . . . 19
accel_metric_z_angle_change . . . 20
allan_nonoverlap . . . 21
as_accel_tbl . . . 22
attach_qc . . . 23
clip_bundle . . . 24
clip_dynamic_stages . . . 25
common-data-format . . . 27
compute_accel_summaries . . . 27
compute_ai_noise_var_from_static . . . 34
dm_apply_vm_calibration_bundle . . . 35
dm_apply_vm_calibration_tbl . . . 36
dm_fit_vm_calibration . . . 37
dynamic_metrology_report . . . 38
feature-comparison . . . 42
filter_accel_tbl . . . 43
getting-started . . . 44
implementation-guide . . . 45
invalidate_qc . . . 48
measurement-definitions . . . 48
m_dynamic_response . . . 49
m_load_sample_data . . . 51
native-recording-examples . . . 52
parse_raw_auto . . . 54
plot_accel_clip . . . 56
plot_processed_metric . . . 57
plot_static_allan . . . 57
plot_static_drift_vm . . . 58
qc_accel_tbl . . . 59
static_1hz_means . . . 60
static_allan . . . 61
static_allan_minima_wide . . . 62
static_axis_noise_metrics . . . 63

static_clip 64
static_drift 65
static_metrology_from_tbl 65
static_noise_summary_from_tbl 67
static_offset_vm_5min 68

Index **70**

`metronautica-package` *Metronautica: Navigate wearable triaxial accelerometer signals into transparent metrology and reports.*

Description

Native wearable triaxial accelerometer parsing, signal diagnostics, clipping, static and dynamic characterisation, filtering and reporting. Developed for the wearable-accelerometer metrology research article under submission for publication; the associated analyses and data are available in WAM: https://github.com/miltheo/wam.

Details

The package provides functions for native decoding, common-format acceleration data, signal diagnostics and selected assessments. Each function documents its input, returned quantities, units and processing settings. `implementation-guide` maps the workflow to the public functions.

Repositories

Package source: https://github.com/miltheo/metronautica. WAM analyses, observations and native-recording catalogue: https://github.com/miltheo/wam. Native recording payloads are separate release attachments.

Start here

`getting-started` provides a complete offline example; `native-recording-examples` covers stationary and ten-stage orbital-shaker recordings.

Standalone counts

The published-filter and custom-cutoff count function `accel_metric_agcounts` is documented separately. The AG selector is disabled in compute_accel_summaries. GT3X parsing defaults to native precision, with an explicit legacy_3dp mode.

Scope

Functions calculate features and metrological outcomes for supplied recordings and selected intervals. Cohort-level inference and inter-device equivalence testing are provided by the linked WAM analyses. Calibration, filtering and analysis settings are explicit caller choices. Dynamic clipping requires explicit stepped-RPM schedule parameters; the native-recording example supplies ten six-minute stages from 50 to 500 RPM.

Workflow

Load a sample or supply a recording, call parse_raw_auto, inspect metadata and diagnostics, select the intended interval, then call the required metrology or feature functions. Use static_1hz_means, static_drift, static_allan and the direct accel_metric_* functions when individual outcomes are needed. Static_metrology_from_tbl and dynamic_metrology_report remain optional combined assessments.

See Also

- implementation-guide
- native-recording-examples
- m_load_sample_data
- parse_raw_auto
- qc_accel_tbl
- static_metrology_from_tbl
- dynamic_metrology_report
- compute_accel_summaries

`accel_meta` *Read acceleration metadata*

Description

Return metadata without modifying the signal.

Usage

```
accel_meta(x)
```

Arguments

x	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.

Value

The `meta` attribute, normally a list; `NULL` if absent.

Implementation

Implemented by `accel_meta` in `R/m_parsers.R`. See implementation-guide for the framework and function map.

See Also

metronautica-package

Examples

```
t0 <- as.POSIXct("2026-01-01 00:00:00", tz = "UTC")
fs <- 100
s <- (0:5999) / fs
meta <- list(acquisition = list(fs_derived_hz = fs,
    fs_nominal_hz = fs, full_scale_range_g = 16))
x <- as_accel_tbl(t0 + s, 0.1 * sin(2 * pi * 2 * s),
    0.1 * cos(2 * pi * 2 * s), 1 + 0.1 * sin(2 * pi * s), meta)
accel_meta(x)$acquisition
```

---

accel_metric_agcounts *Compute ActiGraph-compatible or modified activity counts*

Description

Standalone activity-count calculation with the published coefficient-defined ActiGraph filter by default, explicit custom bandpass cutoffs, and auditable treatment of missing samples and boundaries.

Usage

```
accel_metric_agcounts(tbl, epoch_s = 1, fs_hz,
    lower_hz = NULL, upper_hz = NULL, filter_order = 4,
    lfe = FALSE, gap_policy = c("error", "zero", "carry_forward"),
    vm_round = TRUE, start_time = NULL, end_time = NULL,
    boundary = c("error", "pad"), pad_value = NULL,
    filter_guard_s = 60)
```

Arguments

tbl	Common-format data frame/tibble, parser bundle with data, or clip bundle with full. Required columns are POSIXct time and finite numeric ax_g, ay_g, az_g in g. Rows must be strictly time ordered on the supplied nominal sampling grid. Observed zero-valued samples are retained as observations.
epoch_s	Positive whole epoch duration in seconds. Axis counts are accumulated first; vector magnitude is calculated afterwards. Incomplete trailing raw epochs are discarded before rate conversion and the discarded sample count is recorded.
fs_hz	Nominal recording frequency in Hz, supplied explicitly. Supported rates are 30, 40, 50, 60, 70, 80, 90, 100, 32, 64, 128 and 256 Hz. A frequency estimated across recording gaps is not a substitute for the acquisition setting.
lower_hz, upper_hz	Both NULL retain the exact published filter coefficients; a nominal cutoff pair is not substituted for that response. Supplying both selects a modified causal Butterworth bandpass. These custom frequencies are the lower and upper -3-dB points, satisfying 0 < lower_hz < upper_hz < min(15, fs_hz/2). The historical internal processing rate is 30 Hz.
filter_order	Butterworth prototype order, integer 1 to 8; the bandpass transfer function has twice this order. Default 4. Ignored for the published filter, whose coefficient-defined order remains unchanged. Numerically unstable custom designs are rejected. Custom peak gain is matched to the published filter; subsequent historical scaling and count operations are retained.

`lfe` FALSE uses the Normal-filter threshold rule: rectify, suppress values below 4, saturate at 128 and floor. TRUE selects the published low-frequency-extension threshold rule: suppress values below 1, use floor(value)-1 for 1 <= value < 4, and otherwise floor after saturation at 128. This changes the threshold rule, not the bandpass coefficients. Available real-data AGD validation used FALSE.

`gap_policy` Default error rejects unobserved interior sample positions. zero inserts zero acceleration, while carry_forward repeats the last observed triaxial sample. These are explicit computational conventions, not evidence of the missing physical acceleration. NA/Inf axis values are rejected, not automatically reinterpreted as gaps.

`vm_round` TRUE rounds sqrt(Axis1 squared + Axis2 squared + Axis3 squared) to the nearest integer, as in the R comparison implementation. FALSE retains the unrounded vector magnitude. Summing one-second vector magnitudes is not equivalent to accumulating axes before taking their magnitude.

`start_time`, `end_time`
Optional single POSIXct interval bounds; start is inclusive and end exclusive. Defaults are the first supplied sample and one nominal sample interval after the last. Epochs start at the resolved start_time, without implicit rounding to a civil-clock boundary. Native recording start and first observed sample are recorded separately when parser metadata is available. Process a continuous interval before selecting derived epochs when filter history is required.

`boundary` Default error rejects requested bounds extending beyond observed coverage. pad permits only explicitly requested leading/trailing sample padding, with pad_value required. The function never automatically extends to a native header's start/stop time. Padding does not establish vendor compatibility.

`pad_value` Three finite acceleration values in g for X, Y, Z, used only with boundary = "pad". Both requested ends use these explicit values. The number and location of padded samples are reported. No constant-first-sample startup padding is inserted implicitly.

`filter_guard_s` Non-negative flagging interval after an imputed sample or initialisation, default 60 seconds. Power-of-two conversion additionally flags the preceding second around imputation and the final second of finite-record processing because its FIR stage is centred. This changes flags only, not counts. It is a user-selected processing-context guard, not a validated settling-time guarantee or measurement uncertainty bound; very low custom cutoffs can require a longer guard.

Details

The published path converts each axis to 30 Hz using the rate-specific historical method, including the separate power-of-two path; rounds the converted acceleration to three decimals; applies the coefficient-defined causal filter with steady-state initialisation; rescales by (3/4096)/(2.6/256) times 237.5; rectifies, thresholds, saturates and quantises; averages and floors groups of three samples to 10 Hz; then accumulates epochs. Published finite-record behaviour of the power-of-two converter is retained. Custom filtering replaces only the bandpass stage. It is identified as modified counts and does not inherit validation of published behavioural thresholds.

The implementation uses supplied acceleration precision without reconstructing discarded digits. For GT3X inputs, parse_raw_auto uses native precision by default; precision = "legacy_3dp" explicitly retains the earlier read.gt3x rounded conversion. A manually rounded table remains rounded. Source timestamps and gaps are not repaired by parsing.

This function is available separately from compute_accel_summaries. The AG selector remains disabled, so existing summary pipelines are not silently changed. Neither agcounts nor Python is a runtime dependency.

Value

A tibble with time, Axis1 (input Y), Axis2 (input X), Axis3 (input Z), Vector.Magnitude and value (the same vector magnitude). Additional columns input_samples, missing_samples and padded_samples count nominal positions within each output epoch. imputed identifies epochs containing inserted values; affected_by_imputation also includes the filter guard; boundary_affected identifies initialisation/finite-record context. These flags describe insertions by this count call, not values imputed before input was supplied. Counts are not replaced by NA when an explicit fill policy is chosen: inspect these flags before interpretation. The settings attribute records the reference, exact coefficients, filter choice, precision provenance where available, interval, gaps, padding, discarded samples, rounding and quality conventions.

Validation scope

The original experiment included 89 native/AGD recording pairs: 45 static recordings tested in three 30-minute windows each and 44 secondary-shaker recordings over available raw coverage. The default matched the pinned official Python calculation exactly for 3,763,659 paired one-second axis values. Exact ActiLife export reproduction additionally depended on native precision, startup context and missing-sample conventions; it was not universal across all recording transitions. This is calculation validation, not validation of device interchangeability or behavioural thresholds. The package documentation includes the full agreement and coverage tables.

Implementation

Implemented in R/m_agcounts.R. Standalone count calculation.

References

Neishabouri et al. (2022). Quantification of acceleration as activity counts in ActiGraph wearable. Scientific Reports 12, 11958. doi:10.1038/s4159802216003x.

Published coefficients: ActiGraph/agcounts, GPL-3.0, commit 3137c9b9a26539ae2d9645d396586fe15faece59. https://github.com/actigraph/agcounts. Numerical coefficients are acknowledged from that source; the R implementation is provided by Metronautica.

See Also

parse_raw_auto, qc_accel_tbl, compute_accel_summaries

Examples

```
fs <- 100
t <- (0:5999) / fs
x <- data.frame(time = as.POSIXct("2025-01-01", tz = "UTC") + t,
    ax_g = 0.5 * sin(2*pi*2*t), ay_g = 0.2 * cos(2*pi*t),
    az_g = rep(1, length(t)))
counts <- accel_metric_agcounts(x, epoch_s = 5, fs_hz = fs)
head(counts)
attr(counts, "settings")$filter
modified <- accel_metric_agcounts(x, epoch_s = 5, fs_hz = fs,
    lower_hz = 0.69, upper_hz = 10)
attr(modified, "settings")$algorithm
```

`accel_metric_ai` *Compute Activity Index*

Description

Compute Activity Index.

Usage

```
accel_metric_ai(tbl, epoch_s, fs_hz, anchor = "center", ai_noise_var
    = NULL, ai_noise_mode = c("supplied", "window_start",
    "quiet_segments"),
    ai_noise_window_s = 30, ai_quiet_quantile = 0.1, ai_scale =
    c("absolute", "relative"), ai_epoch_summary = c("sum", "mean"))
```

Arguments

`tbl`	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full. Input conversion preserves values and metadata.
`epoch_s`	Positive output-epoch duration in seconds. Samples are grouped by elapsed time from the first sample; the final partial epoch is retained. Different epoch lengths define different processing pipelines.
`fs_hz`	Sampling frequency in Hz. Used to convert durations to sample windows or sums to time integrals. Supply one finite positive scalar. A NULL estimate is available only where shown in Usage.
`anchor`	Output timestamp selected from the one-second bin starts: first for start, last for end, or the middle bin start (ceiling of half the bin count) for center. This differs from sample-level anchors of ENMO/MAD and does not change the calculated values.
`ai_noise_var`	Mean per-axis stationary noise variance in g squared. Required by default when AI is requested; estimate with compute_ai_noise_var_from_static. An explicit zero requests no noise subtraction.
`ai_noise_mode`	Default supplied requires ai_noise_var. Explicit window_start or quiet_segments opts into estimation when ai_noise_var is NULL. The initial window must be stationary; quiet selection does not establish stationarity. Failed estimation errors rather than substituting zero. A supplied variance takes precedence.
`ai_noise_window_s`	Initial noise-window duration in seconds; used only for window_start estimation.
`ai_quiet_quantile`	Quantile strictly between zero and one for quiet_segments selection.
`ai_scale`	Absolute Activity Index in g (default) or dimensionless noise-relative Activity Index.
`ai_epoch_summary`	Sum (default) or mean of the one-second Activity Index values within each output epoch.

Details

Calculate sample variance per axis within one-second windows, average the three variances, subtract the stationary mean-axis noise variance, truncate negative excess to zero and take the square root. The absolute index is in g. Relative scaling divides excess variance by noise variance before the square root; zero noise variance gives undefined relative values. Aggregate the one-second indices by the selected sum or mean. See `measurement-definitions` for calculation definitions. No additional device calibration is performed by choosing a feature.

Value

A tibble with `epoch_id`, `time` and `value`. Absolute output is in g; relative output is dimensionless. Sum and mean refer to the one-second indices, not acceleration samples. Timestamps select the first, last or middle one-second bin start within each output epoch. The `settings` attribute records noise variance/source, estimation settings, scale, aggregation, rate and duration; `ai_noise_var_used` also records the variance in g squared.

Implementation

Implemented by `accel_metric_ai` in `R/m_accel_summary_suite.R`. See `implementation-guide` for the framework and function map.

References

Bai et al. (2016), doi:10.1371/journal.pone.0160644.

See Also

`metronautica-package`

`accel_metric_enmo` *Compute Euclidean norm minus one*

Description

Compute Euclidean norm minus one.

Usage

```
accel_metric_enmo(tbl, epoch_s, anchor = "center", out_unit =
    c("mg", "g"))
```

Arguments

`tbl`	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full. Input conversion preserves values and metadata.
`epoch_s`	Positive output-epoch duration in seconds. Samples are grouped by elapsed time from the first sample; the final partial epoch is retained. Different epoch lengths define different processing pipelines.

anchor	Output timestamp: start selects the first observed sample in an epoch, end the last, and center the middle observed sample (ceiling of half the epoch sample count). This labels the result without changing values.
out_unit	Output acceleration unit: mg multiplies the epoch result by 1000; g retains the input scale.

Details

Compute max(VM - 1, 0) for every sample, then average within each epoch. Rectification precedes averaging; this is the ENMO definition. See measurement-definitions for calculation definitions. No additional device calibration is performed by choosing a feature.

Value

A tibble with time (the selected observed-sample epoch label), epoch_id (integer elapsed-time bin starting at one), and value (g or mg according to the selected output unit).

Implementation

Implemented by accel_metric_enmo in R/m_accel_summary_suite.R. See implementation-guide for the framework and function map.

See Also

metronautica-package

Examples

```
path <- system.file("extdata", "example_motion.csv",
    package = "metronautica", mustWork = TRUE)
x <- parse_raw_auto(path)
out <- accel_metric_enmo(x, epoch_s = 5)
head(out)
```

---

accel_metric_mad *Compute mean amplitude deviation*

Description

Compute mean amplitude deviation.

Usage

```
accel_metric_mad(tbl, epoch_s, anchor = "center", out_unit = c("mg",
    "g"))
```

Arguments

tbl	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full. Input conversion preserves values and metadata.
epoch_s	Positive output-epoch duration in seconds. Samples are grouped by elapsed time from the first sample; the final partial epoch is retained. Different epoch lengths define different processing pipelines.
anchor	Output timestamp: start selects the first observed sample in an epoch, end the last, and center the middle observed sample (ceiling of half the epoch sample count). This labels the result without changing values.
out_unit	Output acceleration unit: mg multiplies the epoch result by 1000; g retains the input scale.

Details

Within each epoch, calculate VM and average its absolute deviations from that epoch's VM mean. This is the mean amplitude deviation definition. See measurement-definitions for calculation definitions. No additional device calibration is performed by choosing a feature.

Value

A tibble with time (the selected observed-sample epoch label), epoch_id (integer elapsed-time bin starting at one), and value (g or mg according to the selected output unit).

Implementation

Implemented by accel_metric_mad in R/m_accel_summary_suite.R. See implementation-guide for the framework and function map.

References

Vaha-Ypya et al. (2015), A universal, accurate intensity-based classification of different physical activities using raw data of accelerometer. doi:10.1111/cpf.12127.

See Also

metronautica-package

---

accel_metric_mims *Compute MIMS-unit*

Description

Compute MIMS-unit.

Usage

```
accel_metric_mims(tbl, epoch_s, dynamic_range_g, mims_filter_type =
    "butter", mims_cutoffs = c(0.2, 5), mims_use_extrapolation =
    TRUE, mims_use_filtering = TRUE, mims_combination = "sum",
    mims_allow_truncation = TRUE)
```

Arguments

tbl	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full. Input conversion preserves values and metadata.
epoch_s	Positive epoch duration in seconds, passed as the upstream MIMS epoch setting. MIMSunit controls epoch boundaries, timestamps and treatment of partial epochs; this helper does not impose the common elapsed-sample epoch index.
dynamic_range_g	Peak-to-peak configured axis measurement span in g: 16 means -8 to +8 g. The summary function uses valid acquisition metadata when NULL; MIMS fails if neither is available. The direct MIMS function requires this argument. No range is inferred from observed values.
mims_filter_type	Filter family passed to MIMSunit::custom_mims_unit: butter (default), ellip or bessel as supported upstream. Its interpretation belongs to that implementation; select together with mims_cutoffs.
mims_cutoffs	Filter cutoffs in Hz; default c(0.2, 5). Use c(0.2, 10) explicitly for the altered passband.
mims_use_extrapolation	Logical passed to MIMSunit. TRUE enables its saturation/extrapolation step; FALSE skips that step while the upstream algorithm still interpolates to its processing grid. Does not mean extrapolating behavioural time.
mims_use_filtering	Logical; enable the specified internal MIMS band-pass processing. FALSE bypasses that internal filter, creating a different feature pipeline.
mims_combination	Axis-combination method passed to MIMSunit: sum adds axis summaries; vm combines them as a vector magnitude. The default is sum.
mims_allow_truncation	Logical; permit the upstream MIMS implementation to set sufficiently small summary values to zero. This is an algorithmic small-value operation, not clipping native recordings.

Details

Call MIMSunit::custom_mims_unit with triaxial g inputs, configured axis limits and the selected internal processing. Retain its timestamps and combined MIMS output. Default 0.2-5 Hz and explicit altered 0.2-10 Hz are different pipelines; resolved settings and dependency version are returned. See measurement-definitions for calculation definitions. No additional device calibration is performed by choosing a feature.

Value

A tibble with time (upstream MIMS epoch labels), epoch_id (sequential row number), and value (the combined MIMS-unit output per epoch). These values are not g or mg. The settings attribute records measurement span/limits, filter and cutoffs, extrapolation, truncation, combination, duration and MIMSunit version. Upstream epoch boundaries and partial-epoch handling are retained.

Implementation

Implemented by accel_metric_mims in R/m_accel_summary_suite.R. See implementation-guide for the framework and function map.

References

John et al. (2019), An open-source monitor-independent movement summary for accelerometer data processing. doi:10.1123/jmpb.20180068.

See Also

metronautica-package

`accel_metric_odba` *Compute dynamic body acceleration*

Description

Compute dynamic body acceleration.

Usage

```
accel_metric_odba(tbl, epoch_s, fs_hz, anchor = "center", output =
    c("ODBA", "VEDBA", "PDBA"), static_dynamic_method =
    c("centered_running_mean", "low_pass_static_filter",
    "high_pass_dynamic_filter"), stroke_window_rule = c("fixed_3s",
    "custom_window_s", "stroke_period_1x", "stroke_period_2x",
    "sensitivity_windows"), running_mean_window_s = 3,
    stroke_period_s = NA_real_, lowpass_cutoff_hz = 0.2,
    highpass_cutoff_hz = 0.2, filter_type = "butterworth",
    filter_order = 4, pdba_axes = c("x", "y"), out_unit = c("mg",
    "g"), edge_handling = "partial_centered_window",
    missing_data_policy = "mean_na_rm")
```

Arguments

`tbl`	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full. Input conversion preserves values and metadata.
`epoch_s`	Positive output-epoch duration in seconds. Samples are grouped by elapsed time from the first sample; the final partial epoch is retained. Different epoch lengths define different processing pipelines.
`fs_hz`	Sampling frequency in Hz. Used to convert durations to sample windows or sums to time integrals. Supply one finite positive scalar. A NULL estimate is available only where shown in Usage.
`anchor`	Output timestamp: start selects the first observed sample in an epoch, end the last, and center the middle observed sample (ceiling of half the epoch sample count). This labels the result without changing values.
`output`	ODBA sums absolute dynamic components across all three axes; VEDBA uses their Euclidean norm; PDBA sums the selected absolute components. Each is averaged within the output epoch.
`static_dynamic_method`	Axis separation method. centered_running_mean subtracts the centred local axis mean; low_pass_static_filter subtracts a low-pass axis estimate; high_pass_dynamic_filter uses high-pass-filtered axes directly.

`stroke_window_rule`
: Window selection for running-mean separation. fixed_3s uses three seconds; custom_window_s uses running_mean_window_s; stroke_period_1x/2x use max(3, stroke_period_s) or max(3, 2*stroke_period_s). sensitivity_windows uses the requested main window and permits a separate sensitivity summary.

`running_mean_window_s`
: Requested centred averaging-window duration in seconds. Used by custom_window_s and the main result of sensitivity_windows; fixed_3s overrides it. Rounded to samples using fs_hz.

`stroke_period_s`
: Positive representative stroke/movement-cycle period in seconds, required for stroke_period_1x/2x. The function does not estimate a movement period.

`lowpass_cutoff_hz`
: Low-pass cutoff in Hz when static_dynamic_method is low_pass_static_filter. Filter each axis and subtract that static estimate before forming the feature.

`highpass_cutoff_hz`
: High-pass cutoff in Hz when static_dynamic_method is high_pass_dynamic_filter. The filtered axes are the dynamic components.

`filter_type`
: Implementation identifier: butterworth is supported for low-/high-pass separation. This is not an automatic choice among filter families.

`filter_order`
: Order passed to signal::butter for the forward/reverse separation filter. The double-pass frequency response differs from a single pass.

`pdba_axes`
: Axis names selected for PDBA, from x, y and z. Default x/y sums their absolute dynamic components. This option does not change ODBA or VEDBA.

`out_unit`
: Output acceleration unit: mg multiplies the epoch result by 1000; g retains the input scale.

`edge_handling`
: Supported rule partial_centered_window: near signal boundaries, average only the available samples within the centred running window; no external padding is added.

`missing_data_policy`
: Supported rule mean_na_rm: ignore missing observations when calculating local means and epoch means; all-missing estimates remain undefined. This is not gap repair.

Details

Separate static/dynamic axes by the selected implemented method, form ODBA, VEDBA or PDBA, then average within epochs. The centred-window method uses partial available windows at boundaries. A different separation window/filter defines a different pipeline. See `measurement-definitions` for calculation definitions. No additional device calibration is performed by choosing a feature.

Value

A tibble with `time` (the selected observed-sample epoch label), `epoch_id` (integer elapsed-time bin starting at one), and `value` (g or mg according to the selected output unit).

Implementation

Implemented by `accel_metric_odba` in `R/m_accel_summary_suite.R`. See `implementation-guide` for the framework and function map.

Output settings

The `odba_settings` attribute records the selected metric, separation method, requested/resolved window duration, cycle-period rule, filter settings, selected axes, output unit and missing-data/edge policies. The direct helper returns one selected metric, not a list of multiple features. The summary function provides separately requested ODBA, VEDBA and PDBA columns.

See Also

`metronautica-package`

`accel_metric_rocam` *Compute rate of change of acceleration magnitude*

Description

Compute rate of change of acceleration magnitude.

Usage

```
accel_metric_rocam(tbl, epoch_s, fs_hz, anchor = "center", median_s
    = 1)
```

Arguments

`tbl`	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full. Input conversion preserves values and metadata.
`epoch_s`	Positive output-epoch duration in seconds. Samples are grouped by elapsed time from the first sample; the final partial epoch is retained. Different epoch lengths define different processing pipelines.
`fs_hz`	Sampling frequency in Hz. Used to convert durations to sample windows or sums to time integrals. Supply one finite positive scalar. A NULL estimate is available only where shown in Usage.
`anchor`	Output timestamp: start selects the first observed sample in an epoch, end the last, and center the middle observed sample (ceiling of half the epoch sample count). This labels the result without changing values.
`median_s`	Running-median duration in seconds; converted using fs_hz to an odd sample count of at least three. Used to smooth the change or orientation series before epoch summarisation.

Details

Take the Euclidean norm of successive triaxial sample differences, smooth the difference series by a running median, then average within epochs. There is no division by elapsed time or one-second resampling. The retained outcome is g/sample change, not g/s. See `measurement-definitions` for calculation definitions. No additional device calibration is performed by choosing a feature.

Value

A tibble with `time` (the selected observed-sample epoch label), `epoch_id` (integer elapsed-time bin starting at one), and `value` (g per successive-sample change, without division by elapsed time).

Implementation

Implemented by accel_metric_rocam in R/m_accel_summary_suite.R. See implementation-guide for the framework and function map.

See Also

metronautica-package

---

accel_metric_svmbp *Compute band-pass vector magnitude*

Description

Compute band-pass vector magnitude.

Usage

```
accel_metric_svmbp(tbl, epoch_s, fs_hz, anchor = "center",
    cutoffs_hz = c(0.2, 15), order = 4, output = c("mean_mg",
    "mean_g", "sum_mg", "sum_g", "g_seconds"), nyquist_action =
    c("stop", "clip"), quiet = FALSE)
```

Arguments

tbl	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full. Input conversion preserves values and metadata.
epoch_s	Positive output-epoch duration in seconds. Samples are grouped by elapsed time from the first sample; the final partial epoch is retained. Different epoch lengths define different processing pipelines.
fs_hz	Sampling frequency in Hz. Used to convert durations to sample windows or sums to time integrals. Supply one finite positive scalar. A NULL estimate is available only where shown in Usage.
anchor	Output timestamp: start selects the first observed sample in an epoch, end the last, and center the middle observed sample (ceiling of half the epoch sample count). This labels the result without changing values.
cutoffs_hz	Lower and upper axis band-pass cutoffs in Hz, applied before vector-magnitude recombination. The upper cutoff must lie below fs_hz/2 unless clipping is explicitly selected.
order	Butterworth order for axis band-pass filtering before vector-magnitude recombination; default four with forward/reverse filtering.
output	Band-pass VM aggregation: mean_mg and mean_g return epoch means; sum_mg and sum_g return sample sums; g_seconds divides the g sample sum by fs_hz. Sample sums depend on the sample count.
nyquist_action	Default stop rejects an upper cutoff at or above Nyquist. Explicit clip preserves the historical adjustment to 99 percent of Nyquist.
quiet	Logical; suppress optional progress/information messages. Errors remain enabled and calculations are unchanged.

Details

Filter each acceleration axis using signal::butter and forward/reverse filtfilt, recombine the filtered axes as VM, then aggregate. The requested and actual passband are retained. This is distinct from filtering the already-combined VM or from the compensated standalone filter. See measurement-definitions for calculation definitions. No additional device calibration is performed by choosing a feature.

Value

A tibble with time, epoch_id and value. Units are mg/g for means, mg/g sample sums for sum_mg/sum_g, or g seconds for g_seconds. The settings attribute records requested/applied cutoffs, filter order, sampling frequency, Nyquist policy, output aggregation, epoch duration and anchor.

Implementation

Implemented by accel_metric_svmbp in R/m_accel_summary_suite.R. See implementation-guide for the framework and function map.

See Also

metronautica-package

Examples

```
path <- system.file("extdata", "example_motion.csv",
    package = "metronautica", mustWork = TRUE)
x <- parse_raw_auto(path)
out <- accel_metric_svmbp(x, epoch_s = 5, fs_hz = 100,
    cutoffs_hz = c(.69, 10), quiet = TRUE)
head(out)
```

---

accel_metric_svmg *Compute gravity-subtracted vector magnitude*

Description

Compute gravity-subtracted vector magnitude.

Usage

```
accel_metric_svmg(tbl, epoch_s, fs_hz, anchor = "center", output =
    c("g_seconds", "sample_sum_g", "mean_mg", "mean_g"), rectify =
    TRUE)
```

Arguments

tbl	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full. Input conversion preserves values and metadata.

epoch_s	Positive output-epoch duration in seconds. Samples are grouped by elapsed time from the first sample; the final partial epoch is retained. Different epoch lengths define different processing pipelines.
fs_hz	Sampling frequency in Hz. Used to convert durations to sample windows or sums to time integrals. Supply one finite positive scalar. A NULL estimate is available only where shown in Usage.
anchor	Output timestamp: start selects the first observed sample in an epoch, end the last, and center the middle observed sample (ceiling of half the epoch sample count). This labels the result without changing values.
output	g_seconds integrates the gravity-subtracted series by sum/fs_hz; sample_sum_g returns its sample sum; mean_mg and mean_g return its epoch mean in mg/g.
rectify	TRUE takes abs(VM - 1); FALSE retains the signed VM-minus-1-g series. This differs from the non-negative truncation used by ENMO.

Details

Subtract 1 g from sample VM, optionally take absolute differences, and aggregate by the selected output form. The SVMGS, SVMGSEC and SVMGMEAN variants use the absolute differences; ENMO instead truncates negative differences to zero. See measurement-definitions for calculation definitions. No additional device calibration is performed by choosing a feature.

Value

A tibble with time (the selected observed-sample epoch label), epoch_id (integer elapsed-time bin starting at one), and value (g seconds for g_seconds, g sample sums for sample_sum_g, mg for mean_mg, or g for mean_g).

Implementation

Implemented by accel_metric_svmg in R/m_accel_summary_suite.R. See implementation-guide for the framework and function map.

See Also

metronautica-package

---

accel_metric_vm *Compute vector magnitude*

Description

Compute vector magnitude.

Usage

```
accel_metric_vm(tbl, epoch_s, anchor = "center", summary_fun =
    c("mean", "sd", "max", "p95"), out_unit = c("g", "mg"))
```

Arguments

tbl	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full. Input conversion preserves values and metadata.
epoch_s	Positive output-epoch duration in seconds. Samples are grouped by elapsed time from the first sample; the final partial epoch is retained. Different epoch lengths define different processing pipelines.
anchor	Output timestamp: start selects the first observed sample in an epoch, end the last, and center the middle observed sample (ceiling of half the epoch sample count). This labels the result without changing values.
summary_fun	Within-epoch statistic of sample VM: mean, sample SD, maximum, or the R type-7 95th percentile.
out_unit	g retains the VM statistic; mg multiplies it by 1000.

Details

Compute sample vector magnitude and the selected within-epoch statistic; return g or mg. See measurement-definitions for calculation definitions. No additional device calibration is performed by choosing a feature.

Value

A tibble with time (the selected observed-sample epoch label), epoch_id (integer elapsed-time bin starting at one), and value (g or mg according to the selected output unit).

Implementation

Implemented by accel_metric_vm in R/m_accel_summary_suite.R. See implementation-guide for the framework and function map.

See Also

metronautica-package

---

accel_metric_z_angle *Compute Z-axis angle*

Description

Compute Z-axis angle.

Usage

```
accel_metric_z_angle(tbl, epoch_s, anchor = "center", output =
    c("mean_deg", "sd_deg"))
```

Arguments

tbl	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full. Input conversion preserves values and metadata.
epoch_s	Positive output-epoch duration in seconds. Samples are grouped by elapsed time from the first sample; the final partial epoch is retained. Different epoch lengths define different processing pipelines.
anchor	Output timestamp: start selects the first observed sample in an epoch, end the last, and center the middle observed sample (ceiling of half the epoch sample count). This labels the result without changing values.
output	mean_deg returns the epoch mean Z-angle; sd_deg returns its sample SD. Both are in degrees, using atan(z/sqrt(x^2+y^2)).

Details

For each sample compute atan(z/sqrt(x^2+y^2)) in degrees, then take the selected epoch mean or SD. No low-pass gravity-vector estimate is introduced; this is the full-vector orientation feature. See measurement-definitions for calculation definitions. No additional device calibration is performed by choosing a feature.

Value

A tibble with time (the selected observed-sample epoch label), epoch_id (integer elapsed-time bin starting at one), and value (degrees).

Implementation

Implemented by accel_metric_z_angle in R/m_accel_summary_suite.R. See implementation-guide for the framework and function map.

See Also

metronautica-package

---

accel_metric_z_angle_change

Compute change in Z-axis angle

Description

Compute change in Z-axis angle.

Usage

```
accel_metric_z_angle_change(tbl, epoch_s, fs_hz, anchor = "center",
    median_s = 5, output = c("mean_abs_change_deg",
    "max_abs_change_deg"))
```

Arguments

tbl	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full. Input conversion preserves values and metadata.
epoch_s	Positive output-epoch duration in seconds. Samples are grouped by elapsed time from the first sample; the final partial epoch is retained. Different epoch lengths define different processing pipelines.
fs_hz	Sampling frequency in Hz. Used to convert durations to sample windows or sums to time integrals. Supply one finite positive scalar. A NULL estimate is available only where shown in Usage.
anchor	Output timestamp: start selects the first observed sample in an epoch, end the last, and center the middle observed sample (ceiling of half the epoch sample count). This labels the result without changing values.
median_s	Running-median duration in seconds; converted using fs_hz to an odd sample count of at least three. Used to smooth the change or orientation series before epoch summarisation.
output	mean_abs_change_deg averages successive absolute differences of the smoothed Z-angle within each epoch; max_abs_change_deg returns their maximum. Neither is divided by elapsed time.

Details

Compute full-vector Z-angle, apply the selected running median, then form successive absolute differences. Summarise the differences within epochs. It measures change per successive sample, not angular velocity. See measurement-definitions for calculation definitions. No additional device calibration is performed by choosing a feature.

Value

A tibble with time (the selected observed-sample epoch label), epoch_id (integer elapsed-time bin starting at one), and value (degrees per successive-sample change, without division by elapsed time).

Implementation

Implemented by accel_metric_z_angle_change in R/m_accel_summary_suite.R. See implementation-guide for the framework and function map.

See Also

metronautica-package

---

allan_nonoverlap *Compute non-overlapping Allan deviation of a time series*

Description

Compute non-overlapping Allan deviation of a time series.

Usage

```
allan_nonoverlap(x, fs_hz, tau_s, min_M = 4L)
```

Arguments

x	Numeric one-axis acceleration sequence in g, ordered at the supplied sampling frequency. It is not an acceleration table or parser bundle.
fs_hz	Sampling frequency in Hz. Used to convert durations to sample windows or sums to time integrals. Supply one finite positive scalar. A NULL estimate is available only where shown in Usage.
tau_s	Requested Allan averaging times in seconds. Each is rounded to an integer number of samples; duplicate sample counts are removed and unsupported long windows are omitted.
min_M	Minimum number of non-overlapping clusters required at a candidate averaging time. Default four; smaller supported averaging times provide more adjacent-cluster differences.

Value

A tibble containing tau in seconds and adev in the input signal unit.

Method

Convert tau_s to rounded sample counts and divide the sequence into complete non-overlapping clusters. Compute each cluster mean, then sqrt(mean(diff(cluster_means)^2)/2). Omit unsupported cluster lengths and trailing incomplete clusters. Returned tau is the actual sample count divided by fs_hz.

Implementation

Implemented by `allan_nonoverlap` in `R/m_static.R`. See implementation-guide for the framework and function map.

See Also

metronautica-package

---

as_accel_tbl *Construct a common-format acceleration table*

Description

Construct an acceleration table and retain its metadata. Parsing attaches diagnostics by default; they can also be attached explicitly.

Usage

```
as_accel_tbl(time, ax_g, ay_g, az_g, meta = list())
```

Arguments

time	POSIXct sample timestamps. UTC is the common-format convention; existing instants and timezone attributes are retained.
ax_g	Numeric X-axis acceleration in g.
ay_g	Numeric Y-axis acceleration in g.
az_g	Numeric Z-axis acceleration in g.
meta	Metadata list; see common-data-format.

Details

Axes must have the same length as time; scalar recycling is rejected. Non-finite observations and timestamp defects are retained for diagnostics. The constructor does not sort, resample, calibrate or infer acceleration units. The constructor preserves supplied metadata without estimating acquisition settings. The configured axis full-scale range is the peak-to-peak span: 16 g denotes +/-8 g.

Value

An accel_tbl inheriting from tbl_df, with exactly time, ax_g, ay_g, az_g and a meta attribute.

Implementation

Implemented by as_accel_tbl in R/m_parsers.R. See implementation-guide for the framework and function map.

See Also

metronautica-package

Examples

```
t0 <- as.POSIXct("2026-01-01 00:00:00", tz = "UTC")
fs <- 100
s <- (0:5999) / fs
meta <- list(acquisition = list(fs_derived_hz = fs,
    fs_nominal_hz = fs, full_scale_range_g = 16))
x <- as_accel_tbl(t0 + s, 0.1 * sin(2 * pi * 2 * s),
    0.1 * cos(2 * pi * 2 * s), 1 + 0.1 * sin(2 * pi * s), meta)
class(x)
qc_accel_tbl(x)$n_samples
```

---

attach_qc *Attach signal diagnostics to recording metadata*

Description

Attach signal diagnostics to recording metadata.

Usage

```
attach_qc(x, scope = "whole parsed recording")
```

Arguments

x	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.
scope	Text describing the recording interval to which the diagnostics apply.

Value

The same input container, with primary-five diagnostics in meta$processing$qc. Bundle and data metadata are synchronised.

Implementation

Implemented by `attach_qc` in `R/m_qc.R`. See `implementation-guide` for the framework and function map.

See Also

`metronautica-package`

`clip_bundle` *Select a time interval with provenance*

Description

Select an inclusive time interval from an acceleration table or parser bundle.

Usage

```
clip_bundle(obj, start_time = NULL, duration_s = NULL, end_time =
    NULL, trim_pad_s = 0, keep_qc = FALSE)
```

Arguments

obj	An acceleration table, parser bundle with `data`, or dynamic bundle with `full`; see Stage bundles for the scope of clipping.
start_time	Inclusive UTC bounds (POSIXct or parseable text). Supply a start with an end or duration, or an end with a duration.
duration_s	Duration in seconds; used to infer a missing bound. Explicit start and end take precedence.
end_time	Inclusive UTC bounds (POSIXct or parseable text). Supply a start with an end or duration, or an end with a duration.
trim_pad_s	Seconds added before the start and after the end; a negative value contracts the interval.
keep_qc	Retain inherited diagnostics if TRUE. They are not recomputed and may describe the original recording.

Details

Both endpoints are included. A 10-s interval sampled at 100 Hz can therefore contain 1001 samples. This differs deliberately from the exclusive end used by static and dynamic stage clipping. The input is unchanged. Invalid or empty intervals raise errors. Recompute `qc_accel_tbl()` for the selected interval.

Value

The same object form as the input. Acquisition bounds are updated and a clipping event is appended to `meta$processing$history`. Inherited QC and a top-level bundle QC entry are removed by default.

Implementation

Implemented by `clip_bundle` in `R/m_accel_clipper.R`. See implementation-guide for the framework and function map.

Stage bundles

A dynamic bundle with `full` is also accepted. The selected full recording is clipped, while existing `stages` and `schedule` are retained unchanged. Construct stages again from the intended clipped recording before stage-based metrology. This differs from `dm_apply_vm_calibration_bundle`, which explicitly transforms every stored signal.

See Also

metronautica-package

`clip_dynamic_stages` *Select trimmed excitation stages*

Description

Build a timed excitation schedule and select trimmed, end-exclusive stages.

Usage

```
clip_dynamic_stages(x, start_utc, n_stages, stage_min,
    buffer_min, rpm_start, rpm_step, trim_start_min, trim_end_min,
    keep_empty = FALSE, verbose = FALSE,
    time_offset_s = 0)
```

Arguments

`x`	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.
`start_utc`	UTC window start; POSIXct or parseable text.
`n_stages`	Required positive integer number of excitation stages; not inferred from the data.
`stage_min`	Required raw duration of each stage in minutes, before trimming.

buffer_min	Required non-negative interval in minutes between raw stages. It is excluded from the stage tables but retained within the complete assessment span.
rpm_start	Required numerical RPM label assigned to the first stage; it is a supplied excitation condition, not a measured frequency.
rpm_step	Required increment in the RPM label between consecutive stages. The clipper assumes this regular schedule.
trim_start_min	Required non-negative minutes removed from the beginning of every raw stage. Start/end trims must sum to less than stage_min.
trim_end_min	Required non-negative minutes removed from the end of every raw stage. Trimming changes the analysed interval, not the native file.
keep_empty	Retain empty tables if TRUE; otherwise store NULL at the corresponding stage position.
verbose	Logical; print processing messages. Does not change calculations.
time_offset_s	Additive schedule-anchor shift in seconds; sample timestamps are unchanged.

Details

All schedule parameters must be supplied explicitly for the experiment; this helper does not infer an excitation schedule from the recording. The input is sorted by timestamp within this operation; inspect whole-recording QC beforehand. Each stage spans $[start, end)$. The first raw stage starts at `start_utc` + `time_offset_s`; consecutive starts are separated by `stage_min` + `buffer_min`. Trims must total less than the raw stage duration. Missing stages remain positionally identifiable; resolve them before requesting paired metrology.

Value

A list with `full` (untrimmed full-assessment table), `stages` (named list, e.g. `rpm_50`), `schedule` (raw and trimmed UTC bounds, settings and labels), and `meta`. Each clip has freshly computed QC.

Implementation

Implemented by `clip_dynamic_stages` in `R/m_dynamic.R`. See implementation-guide for the framework and function map.

Processing order

Apply any intended axis filtering to the recording before constructing stages. Filtering an existing dynamic bundle with `filter_accel_tbl` changes only its full recording, not its stored stage tables. Stage-based response uses `stages`, while some whole-recording diagnostics use `full`; these signals must represent the intended processing state.

See Also

metronautica-package

common-data-format *Acceleration data and metadata contract*

Description

The common data format shared by parsing, metrology and feature functions.

Details

An `accel_tbl` is a tibble with exactly four columns: `time` (POSIXct), `ax_g`, `ay_g`, `az_g` (numeric acceleration in g, where 1 g corresponds to 9.80665 m/s squared). UTC is the storage convention. No unit inference is performed by the constructor. Preserve acquisition timestamps, gaps and invalid observations until the corresponding diagnostics have been inspected.

The `meta` attribute is a list. `acquisition` contains available device identifiers, `fs_nominal_hz`, `fs_derived_hz`, `full_scale_range_g`, recording bounds and time provenance. Range denotes the full axis span, not the positive limit. `file` records source information; `processing` records parser/calibration details, QC and history. Additional provenance can be retained. A parser bundle is `list(data = x, meta = accel_meta(x))`; keep these two metadata representations synchronised when constructing bundles externally.

The constructor `as_accel_tbl()` preserves the supplied metadata. It neither attaches QC nor estimates sampling frequency. Call `attach_qc()` explicitly to attach diagnostics; parsing through `parse_raw_auto()` enables QC by default. Parser paths populate available acquisition settings; feature functions resolve a missing rate as documented below. The constructor does not sort or repair the recording. `accel_meta()` returns the attached metadata. Subsetting externally may retain stale whole-recording QC; recompute it or use the provided clipping functions. Filtering invalidates inherited QC. Static/dynamic stage clipping recomputes QC for each selected window.

Feature rate resolution uses an explicit `fs_hz`, then metadata, then timestamps.

Axis filtering with `filter_accel_tbl()` requires `fs_derived_hz`. A nominal or timestamp-derived rate is not an independently validated clock rate. Sample-based filters and index pairing require appropriate regularity/synchronisation beyond the presence of metadata.

See Also

`getting-started`, `metronautica-package`, `as_accel_tbl`, `qc_accel_tbl`, `clip_bundle`

compute_accel_summaries *Compute epoch-level acceleration features*

Description

Compute time-domain summaries through explicitly configured processing pipelines.

Usage

```
compute_accel_summaries(accel_input, measures = c("ENMO", "MAD"),
    epoch_s = 5, epoch_anchor = c("center", "start", "end"), fs_hz =
    NULL, target_fs = NULL, dynamic_range_g = NULL, vm_summary =
    c("mean", "sd", "max", "p95"), vm_unit = c("g", "mg"), enmo_mg =
    TRUE, mad_mg = TRUE, odba_static_dynamic_method =
    c("centered_running_mean", "low_pass_static_filter",
    "high_pass_dynamic_filter"), odba_stroke_window_rule =
    c("fixed_3s", "custom_window_s", "stroke_period_1x",
    "stroke_period_2x", "sensitivity_windows"),
    odba_running_mean_window_s = 3, odba_stroke_period_s = NA_real_,
    odba_lowpass_cutoff_hz = 0.2, odba_highpass_cutoff_hz = 0.2,
    odba_filter_type = "butterworth", odba_filter_order = 4,
    pdba_axes = c("x", "y"), odba_unit = c("mg", "g"),
    odba_sensitivity_windows_s = c(1, 2, 3, 5, 10),
    odba_edge_handling = "partial_centered_window",
    odba_missing_data_policy = "mean_na_rm", svmg_output =
    c("g_seconds", "sample_sum_g", "mean_mg", "mean_g"),
    svmg_rectify = TRUE, svmbp_cutoffs = c(0.2, 15), svmbp_order =
    4, svmbp_output = c("mean_mg", "mean_g", "sum_mg", "sum_g",
    "g_seconds"), svmbp_nyquist_action = c("stop", "clip"),
    rocam_median_s = 1, ai_noise_var = NULL, ai_noise_mode =
    c("supplied", "window_start", "quiet_segments"), ai_noise_window_s = 30,
    ai_quiet_quantile = 0.1, ai_scale = c("absolute", "relative"),
    ai_epoch_summary = c("sum", "mean"), z_angle_output =
    c("mean_deg", "sd_deg"), z_angle_change_median_s = 5,
    z_angle_change_output = c("mean_abs_change_deg",
    "max_abs_change_deg"), mims_filter_type = "butter", mims_cutoffs
    = c(0.2, 5), mims_use_extrapolation = TRUE, mims_use_filtering
    = TRUE, mims_combination = "sum", mims_allow_truncation = TRUE,
    ag_lfe = FALSE, ag_preprocess = c("none", "regularise",
    "paper30", "antialias100"), ag_anchor = c("floor_sec",
    "round_sec"), ag_antialias_cutoff_hz = NULL, ag_filter_order =
    4, quiet = FALSE)
```

Arguments

accel_input	Common-format acceleration table, parser bundle with data, or dynamic clip bundle with full. A plain data frame with the four common columns is also accepted.
measures	Distinct case-insensitive identifiers listed in Feature identifiers, with their direct functions and default outputs.
epoch_s	Positive output-epoch duration in seconds. Samples are grouped by elapsed time from the first sample; the final partial epoch is retained. Different epoch lengths define different processing pipelines.
epoch_anchor	Timestamp assigned to an output epoch: start, end or center selects the first, last or middle observed sample respectively. Some upstream feature implementations retain their own anchors; inspect times before joining features.
fs_hz	Optional explicit input frequency in Hz. Otherwise use valid acquisition metadata, then the median positive timestamp interval. target_fs, when supplied,

controls a separate interpolation step; neither setting establishes regular sampling.

`target_fs` Optional positive target rate in Hz. Linear interpolation is applied when it differs from the input rate; this alone is not an anti-aliasing filter.

`dynamic_range_g`
Peak-to-peak configured axis measurement span in g: 16 means -8 to +8 g. The summary function uses valid acquisition metadata when NULL; MIMS fails if neither is available. The direct MIMS function requires this argument. No range is inferred from observed values.

`vm_summary` Statistic of sample VM within each epoch: mean, sample SD, maximum, or the 95th percentile. All use sqrt(ax_g^2 + ay_g^2 + az_g^2).

`vm_unit` Output unit for the VM statistic: g, or mg after multiplication by 1000. No calibration is implied.

`enmo_mg` Logical; multiply the ENMO epoch mean by 1000 for mg when TRUE; otherwise retain g. Negative VM-minus-1-g samples are truncated before averaging.

`mad_mg` Logical; multiply within-epoch mean absolute deviation of VM by 1000 for mg when TRUE; otherwise retain g.

`odba_static_dynamic_method`
Axis separation method for ODBA/VEDBA/PDBA. Subtract a centred running mean or low-pass static component, or use high-pass-filtered axes directly. These are distinct signal-processing choices.

`odba_stroke_window_rule`
Running-mean window rule: fixed three seconds; supplied custom duration; max(3, one or two supplied stroke periods); or a main-window result plus the specified sensitivity-window summaries.

`odba_running_mean_window_s`
Requested running-mean duration in seconds for custom_window_s or the main sensitivity result. fixed_3s overrides it; the resolved duration is returned in settings$odba.

`odba_stroke_period_s`
Supplied movement-cycle period in seconds for stroke-period rules; not estimated from the signal. A minimum three-second averaging window is retained.

`odba_lowpass_cutoff_hz`
Axis low-pass cutoff in Hz used only for low_pass_static_filter separation; the resulting static estimate is subtracted from the original axis.

`odba_highpass_cutoff_hz`
Axis high-pass cutoff in Hz used only for high_pass_dynamic_filter separation.

`odba_filter_type`
Separation implementation identifier; butterworth is supported. The method/cutoff/order together define the processing pipeline.

`odba_filter_order`
Butterworth order for low-/high-pass dynamic separation. The centred-running-mean method does not use this filter.

`pdba_axes` Axis names selected for PDBA, from x, y and z. Default x/y sums their absolute dynamic components. This option does not change ODBA or VEDBA.

`odba_unit` Output unit for ODBA/VEDBA/PDBA epoch means: mg or g. mg applies a factor of 1000.

`odba_sensitivity_windows_s`
: Vector of alternative running-mean durations in seconds for the sensitivity_windows rule. Results are returned separately in settings$odba$sensitivity_summary; they do not replace the main feature values.

`odba_edge_handling`
: partial_centered_window averages the available part of a centred window at the recording boundaries, rather than padding a stationary component.

`odba_missing_data_policy`
: mean_na_rm omits missing values from means; it does not interpolate gaps or establish sufficient coverage.

`svmg_output`
: Retained compatibility argument; the SVMGS/SVMGSEC/SVMGMEAN identifiers fix their output form.

`svmg_rectify`
: Apply the absolute value to VM minus 1 g for the gravity-subtracted VM family; this differs from ENMO truncation.

`svmbp_cutoffs`
: Custom lower/upper axis band-pass cutoffs in Hz for SVMBP. Named variants fix their cutoffs.

`svmbp_order`
: Butterworth order for axis band-pass filtering before vector-magnitude recombination; default four with forward/reverse filtering.

`svmbp_output`
: Band-pass VM aggregation: mean_mg and mean_g return epoch means; sum_mg and sum_g return sample sums; g_seconds divides the g sample sum by fs_hz. Sample sums depend on the sample count.

`svmbp_nyquist_action`
: Default stop rejects an upper cutoff at or above Nyquist. Explicit clip preserves the historical adjustment to 99 percent of Nyquist.

`rocam_median_s`
: Duration in seconds of the running median on the sample-to-sample triaxial acceleration-change series. Rounded to samples, raised to at least three, and made odd. No division by elapsed time is performed.

`ai_noise_var`
: Mean per-axis stationary noise variance in g squared. Required by default when AI is requested; estimate with compute_ai_noise_var_from_static. An explicit zero requests no noise subtraction.

`ai_noise_mode`
: Default supplied requires ai_noise_var. Explicit window_start or quiet_segments opts into estimation when ai_noise_var is NULL. The initial window must be stationary; quiet selection does not establish stationarity. Failed estimation errors rather than substituting zero. A supplied variance takes precedence.

`ai_noise_window_s`
: Initial noise-window duration in seconds; used only for window_start estimation.

`ai_quiet_quantile`
: Quantile strictly between zero and one for quiet_segments selection.

`ai_scale`
: Absolute Activity Index in g (default) or dimensionless noise-relative Activity Index.

`ai_epoch_summary`
: Sum (default) or mean of the one-second Activity Index values within each output epoch.

`z_angle_output`
: mean_deg returns the epoch mean Z-angle; sd_deg returns its sample SD. Both are in degrees, using atan(z/sqrt(x^2+y^2)).

`z_angle_change_median_s`
: Running-median duration in seconds applied to the Z-angle series before absolute successive differences. Rounded to an odd sample count of at least three.

`z_angle_change_output`
: mean_abs_change_deg averages successive absolute differences of the smoothed Z-angle within each epoch; max_abs_change_deg returns their maximum. Neither is divided by elapsed time.

`mims_filter_type`
: Filter family passed to MIMSunit::custom_mims_unit: butter (default), ellip or bessel as supported upstream. Its interpretation belongs to that implementation; select together with mims_cutoffs.

`mims_cutoffs`
: Filter cutoffs in Hz; default c(0.2, 5). Use c(0.2, 10) explicitly for the altered passband.

`mims_use_extrapolation`
: Logical passed to MIMSunit. TRUE enables its saturation/extrapolation step; FALSE skips that step while the upstream algorithm still interpolates to its processing grid. Does not mean extrapolating behavioural time.

`mims_use_filtering`
: Logical; enable the specified internal MIMS band-pass processing. FALSE bypasses that internal filter, creating a different feature pipeline.

`mims_combination`
: Axis-combination method passed to MIMSunit: sum adds axis summaries; vm combines them as a vector magnitude. The default is sum.

`mims_allow_truncation`
: Logical; permit the upstream MIMS implementation to set sufficiently small summary values to zero. This is an algorithmic small-value operation, not clipping native recordings.

`ag_lfe`
: Retained count-specific compatibility argument. The AG selector is disabled in this summary function; this argument is not used by supported features. For counts use the separate `accel_metric_agcounts` function and its own parameters.

`ag_preprocess`
: Retained count-specific compatibility argument. The AG selector is disabled in this summary function; this argument is not used by supported features. For counts use the separate `accel_metric_agcounts` function and its own parameters.

`ag_anchor`
: Retained count-specific compatibility argument. The AG selector is disabled in this summary function; this argument is not used by supported features. For counts use the separate `accel_metric_agcounts` function and its own parameters.

`ag_antialias_cutoff_hz`
: Retained count-specific compatibility argument. The AG selector is disabled in this summary function; this argument is not used by supported features. For counts use the separate `accel_metric_agcounts` function and its own parameters.

`ag_filter_order`
: Retained count-specific compatibility argument. The AG selector is disabled in this summary function; this argument is not used by supported features. For counts use the separate `accel_metric_agcounts` function and its own parameters.

`quiet`
: Logical; suppress optional progress/information messages. Errors remain enabled and calculations are unchanged.

Details

A feature-processing pipeline includes signal state, algorithm, filtering, sampling treatment and aggregation. The feature examples use five-second epochs; configurable alternatives are distinct pipelines. The input is sorted by timestamp for feature computation. Epochs are indexed by elapsed time from the first sample and the final partial epoch is retained. Feature-specific time anchors are retained from their implementations. Gaps are not automatically screened or repaired; inspect integrity before filtering or feature computation. MIMS requires optional MIMSunit. The AG selector is disabled in this summary function; standalone count computation is available through `accel_metric_agcounts`. MIMS requires a configured peak-to-peak measurement span from an explicit argument or valid metadata; no range is assumed. Activity Index requires a stationary noise variance unless estimation is explicitly selected. Optional packages are never installed during calculation.

Value

Resolved band-pass settings record requested and applied cutoffs, filter order, sampling rate and Nyquist handling. The summary settings also include named metric_units for plotting and a band-pass list for requested SVM band-pass pipelines. The settings list includes dynamic_range_g, dynamic_range_limits_g and dynamic_range_source (argument, metadata or unavailable). When requested, settings$mims records the resolved MIMS configuration and dependency version, and settings$ai records the noise variance, source, applicable estimation options, scale and aggregation. Unrequested feature settings are NULL. An invisible list with `data_wide` (time and one column per measure), `data_long` (time, measure, value), and `settings` (resolved rate, input provenance, dynamic range and selected processing details). No behavioural classification is returned. Save the complete call/settings when reproducing a pipeline; the returned settings do not enumerate every function argument.

Implementation

Implemented by `compute_accel_summaries` in `R/m_accel_summary_suite.R`. See `implementation-guide` for the framework and function map.

Feature identifiers

Identifier	Direct function	Default output
VM	`accel_metric_vm`	Mean, g
ENMO	`accel_metric_enmo`	Mean, mg
MAD	`accel_metric_mad`	Mean, mg
ODBA	`accel_metric_odba`	Mean, mg
VEDBA	`accel_metric_odba`	Mean, mg
PDBA	`accel_metric_odba`	Mean, mg
SVMGS	`accel_metric_svmg`	Sample sum, g
SVMGSEC	`accel_metric_svmg`	Integral, g seconds
SVMGMEAN	`accel_metric_svmg`	Mean, mg
SVMBP	`accel_metric_svmbp`	Mean, mg
SVMBP_02_15	`accel_metric_svmbp`	Mean, mg
SVMBP_069_10	`accel_metric_svmbp`	Mean, mg
ROCAM	`accel_metric_rocam`	Mean change, g
AI	`accel_metric_ai`	Sum of 1-s AI, g
ZANGLE	`accel_metric_z_angle`	Mean, degrees
ZANGLE_CHANGE	`accel_metric_z_angle_change`	Mean change, degrees
MIMS	`accel_metric_mims`	MIMS-units per epoch

Identifiers are case-insensitive. The SVM gravity-subtracted identifiers fix their aggregation; `svmg_rectify = FALSE` instead retains signed differences. SVMBP uses `svmbp_cutoffs`; the named variants fix their cutoffs, while `svmbp_output` still selects their aggregation. VM, ENMO/MAD, body-acceleration, AI and angle alternatives are specified in Arguments and their direct helpers. There are 17 accepted identifiers including the configurable SVMBP selector. Counts use the standalone `accel_metric_agcounts` function; AG is not an available selector.

Epoch timestamps and joins

Sample-level features use elapsed-time bins from the first sample and select the first, last or middle observed sample as their label. AI first computes one-second indices and selects among their bin-start timestamps. MIMS retains upstream timestamps and epoch behaviour. Consequently the same nominal epoch duration does not guarantee identical timestamps across features.

`data_long` retains each feature's own timestamp/value pairs. `data_wide` is a full join on exact timestamps: different labels create separate rows and unpaired NA cells, even when each feature has the same number of epochs. With 20 seconds at 100 Hz and five-second center-labelled epochs, ENMO labels are 2.49, 7.49, 12.49 and 17.49 seconds after the first sample; AI labels are 2, 7, 12 and 17 seconds. Their wide join therefore has eight rows. Inspect labels and interval definitions before aligning or combining features. Do not interpret every unpaired cell as missing acceleration or replace it with zero. The function does not harmonise these anchors.

See Also

`metronautica-package`

Examples

```
# Specify the complete feature pipeline, including aggregation and units.
library(metronautica)
path <- system.file("extdata", "example_motion.csv",
    package = "metronautica", mustWork = TRUE)
x <- parse_raw_auto(path)
configuration <- list(measures = c("VM", "ENMO", "MAD", "SVMBP_069_10"),
    epoch_s = 5, fs_hz = 100, svmbp_nyquist_action = "stop", quiet = TRUE)
five_seconds <- do.call(compute_accel_summaries,
    c(list(accel_input = x), configuration))
five_seconds$data_wide          # VM in g; ENMO, MAD and SVMBP_069_10 in mg

configuration_10s <- configuration
configuration_10s$epoch_s <- 10
ten_seconds <- do.call(compute_accel_summaries,
    c(list(accel_input = x), configuration_10s))
nrow(five_seconds$data_wide)    # 4
nrow(ten_seconds$data_wide)     # 2; aggregation is part of the pipeline

# Other built-in summaries, with an explicit noise variance for Activity Index.
# Here 0.001^2 / 2 is the variance of each stationary sine/cosine fluctuation
# used in the static tutorial; real analyses require their own noise estimate.
other <- compute_accel_summaries(x,
    measures = c("ODBA", "VEDBA", "PDBA", "SVMGS", "SVMGSEC", "SVMGMEAN",
        "ROCAM", "AI", "ZANGLE", "ZANGLE_CHANGE"),
    epoch_s = 5, odba_running_mean_window_s = 3,
```

```
    ai_noise_var = .001^2 / 2, quiet = TRUE)
head(other$data_long)
other$data_long[other$data_long$measure == "AI", ]
# Some feature implementations retain different timestamp anchors. A wide
# full join can therefore have additional rows and unpaired (NA) cells.
# Inspect each feature's timestamps before combining results by time.

# The standalone filter is a separate processing choice.
filtered <- filter_accel_tbl(x, cutoff = 10)
qc_accel_tbl(filtered)[c("n_samples", "median_dt_ms", "invalid_axis")]
# Do not prefilter x merely to request SVMBP_069_10: that feature filters axes.

# Resolved MIMS and AI settings are included when those measures are requested.
# Retain the full call and session for the complete processing record.
output_file <- tempfile(fileext = ".rds")
saveRDS(list(results = five_seconds, configuration = configuration,
    input_metadata = accel_meta(x), session = utils::sessionInfo()), output_file)
names(readRDS(output_file))
unlink(output_file)             # Remove only this example's temporary output
```

`compute_ai_noise_var_from_static` *Estimate stationary variance for the Activity Index*

Description

Estimate stationary variance for the Activity Index.

Usage

```
compute_ai_noise_var_from_static(x, window_s = 1, fs_hz = NULL,
    use_middle_prop = 1)
```

Arguments

`x`	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.
`window_s`	Duration in seconds of each non-overlapping stationary noise window. round(fs_hz * window_s) must be at least two samples. Axis sample variances are averaged within each window, then across retained complete windows.
`fs_hz`	NULL derives Hz from the median finite positive timestamp interval; otherwise use the supplied finite positive Hz to convert window_s to samples.
`use_middle_prop`	Fraction in (0, 1] of the central recording retained before windowing. One uses the complete supplied interval; 0.8 selects its middle 80 percent. No stationarity is inferred.

Value

A list with ai_noise_var (mean per-axis stationary variance, g squared), ai_noise_sd (its square root, g), window_values (per-window variances), n_windows, fs_hz and window_s. Trailing incomplete sample windows are discarded. No finite estimates or fewer than two samples per window raises an error.

Method

For every complete stationary window calculate (var(x)+var(y)+var(z))/3 using sample variances. Average these window variances for the Activity Index noise correction. The estimate differs from the maximum-axis detrended noise floor. The supported table/bundle forms feed the same calculation; no calibration, detrending or stationarity detection is added.

Implementation

Implemented by `compute_ai_noise_var_from_static` in `R/m_accel_summary_suite.R`. See `implementation-guide` for the framework and function map.

See Also

`metronautica-package`

Examples

```
seconds <- (0:5999) / 100
x <- as_accel_tbl(as.POSIXct("2026-01-01", tz = "UTC") + seconds,
    .001 * sin(2*pi*7*seconds), .001 * cos(2*pi*7*seconds),
    1 + .001 * sin(2*pi*11*seconds))
noise <- compute_ai_noise_var_from_static(list(data = x), window_s = 1)
noise[c("ai_noise_var", "ai_noise_sd", "n_windows")]
```

---

dm_apply_vm_calibration_bundle

Apply vector-magnitude calibration to a stage bundle

Description

Apply vector-magnitude calibration to a stage bundle.

Usage

```
dm_apply_vm_calibration_bundle(clips, cal_row)
```

Arguments

`clips`	Acceleration table, parser bundle with data, or dynamic bundle with full and stages. Every present signal is calibrated; NULL stages remain NULL.
`cal_row`	One-row data frame/tibble or named scalar list containing finite numeric vm_calibration_intercept, vm_calibration_slope and vm_calibration_quadratic. Supply zero for the quadratic term of a linear model. If vm_calibration_direction is present it must be ref_on_test. Invalid/ambiguous rows raise an error.

Value

The same table or bundle form. Apply identical coefficients to full/data and each non-NULL stage. Per-table metadata and top-level meta record the operation; inherited QC is removed from all modified signals and the top-level bundle.

Method

Use dm_apply_vm_calibration_tbl for every stored signal. Missing stage entries remain NULL; present stage names and all timestamps remain unchanged. Invalid coefficients fail before a calibrated result is returned. The function neither refits the model nor validates its benefit on independent recordings.

Implementation

Implemented by `dm_apply_vm_calibration_bundle` in `R/m_dynamic.R`. See `implementation-guide` for the framework and function map.

See Also

`metronautica-package`

`dm_apply_vm_calibration_tbl` *Apply vector-magnitude calibration to triaxial samples*

Description

Apply vector-magnitude calibration to triaxial samples.

Usage

```
dm_apply_vm_calibration_tbl(df, cal_row)
```

Arguments

`df`	Acceleration table or parser/clip bundle with POSIXct time and numeric axes in g. Container type and timestamps are preserved.
`cal_row`	One-row data frame/tibble or named scalar list containing finite numeric vm_calibration_intercept, vm_calibration_slope and vm_calibration_quadratic. Supply zero for the quadratic term of a linear model. If vm_calibration_direction is present it must be ref_on_test. Invalid/ambiguous rows raise an error.

Value

The same input container type, with calibrated axes and unchanged timestamps. A parser bundle retains its data/meta structure; a stage bundle also calibrates its stages. Metadata records the applied row and processing event, marks the signal calibrated and removes pre-calibration QC. Call attach_qc to obtain diagnostics for the changed signal.

Method

Map test VM v to max(b0+b1*v+b2*v^2, 0), then multiply every axis at that sample by corrected_VM/v. The original scale fallback of one for zero/undefined VM is retained. This scalar mapping preserves the instantaneous acceleration-vector direction; it is not a general axis-specific calibration. Finite single-row coefficients are required. When a direction is provided it must be ref_on_test. Source decoding provenance remains available; added calibration history is separate.

Implementation

Implemented by `dm_apply_vm_calibration_tbl` in `R/m_dynamic.R`. See `implementation-guide` for the framework and function map.

See Also

`metronautica-package`

`dm_fit_vm_calibration` *Fit a reference-based vector-magnitude calibration*

Description

Fit a reference-based vector-magnitude calibration.

Usage

```
dm_fit_vm_calibration(test_clips, ref_clips, window_s = 5, select_by
    = c("AIC", "AICc", "BIC", "linear"), fit_unit = c("window",
    "sample"), direction = c("ref_on_test", "test_on_ref"))
```

Arguments

`test_clips`	Test-device stage bundle returned by `clip_dynamic_stages`.
`ref_clips`	Reference-device stage bundle with matching stage names.
`window_s`	Duration in seconds of paired-VM averaging bins when fit_unit is window. Index-paired samples are averaged before fitting. Unused for fit_unit = sample.
`select_by`	Select the linear or quadratic candidate using AIC, corrected AIC (AICc), or BIC; `"linear"` forces the linear candidate.
`fit_unit`	window fits paired epoch-mean VM values; sample fits the paired sample values. Pairing is by existing stage/sample-index rules and is not timestamp alignment.
`direction`	ref_on_test fits reference VM as a function of test VM, producing coefficients suitable for application. test_on_ref reverses the regression for inspection and must not be applied as a test-to-reference correction.

Value

A list with summary (one-row coefficients/model-selection results), fit_linear and fit_quadratic (lm objects), chosen_fit and pairs (test_vm/reference_vm fitting data). Insufficient pairs yield no fitted model and NA coefficients; application rejects those coefficients.

Method

Form VM from each matched stage and pair reference observations using the retained evenly spaced sample-index rule. Fit linear and quadratic candidates in the requested regression direction; select by the supplied criterion or force linear. This is within-recording fitted calibration, not independent validation. Apply only ref_on_test coefficients to test recordings.

Implementation

Implemented by `dm_fit_vm_calibration` in `R/m_dynamic.R`. See implementation-guide for the framework and function map.

See Also

metronautica-package

Examples

```
t0 <- as.POSIXct("2026-01-01", tz = "UTC")
seconds <- (0:5999) / 100
test_vm <- 1 + .2 * sin(2 * pi * .17 * seconds) + .003 * seconds
meta <- list(acquisition = list(fs_derived_hz = 100,
    full_scale_range_g = 16))
test <- as_accel_tbl(t0 + seconds, rep(0, 6000), rep(0, 6000),
    test_vm, meta)
reference <- as_accel_tbl(t0 + seconds, rep(0, 6000), rep(0, 6000),
    .02 + 1.1 * test_vm, meta)
stage <- function(x) list(full = x, stages = list(rpm_50 = x),
    meta = accel_meta(x))
fit <- dm_fit_vm_calibration(stage(test), stage(reference),
    select_by = "linear", fit_unit = "window", window_s = 5)
fit$summary[, c("vm_calibration_intercept", "vm_calibration_slope")]
calibrated <- dm_apply_vm_calibration_tbl(attach_qc(test), fit$summary)
is.null(accel_meta(calibrated)$processing$qc)  # Previous QC was removed
calibrated <- attach_qc(calibrated)           # Diagnostics for changed signal
accel_meta(calibrated)$processing$signal_pipeline
```

---

dynamic_metrology_report

Characterise response relative to a reference

Description

Compute paired-stage response and full-range error metrics from explicitly supplied device and reference signals.

Usage

```
dynamic_metrology_report(test_clips, ref_clips, test_axis_fsr_g =
    NULL, ref_axis_fsr_g = NULL, window_s = 5, epoch_time = c("end",
    "center", "start"), compute_entropy = TRUE, entropy_m = 2,
    entropy_r = NULL, entropy_tau = 1, verbose = FALSE, progress =
    TRUE, add_plot = TRUE, inspect_alignment = TRUE, align_hz = 10,
```

```
    align_max_lag_s = 7200, align_by = c("vm", "ax", "ay", "az"),
    align_plot_stages_n = 3, compute_stage_xcorr = FALSE,
    comp_plot_max_n = 20000L, comp_plot_ylim = NULL)
```

Arguments

test_clips	Test-device stage bundle returned by clip_dynamic_stages.
ref_clips	Reference-device stage bundle with matching stage names.
test_axis_fsr_g	A supplied span must be a finite positive scalar; invalid supplied values error rather than falling back to metadata. A valid test span is required for range-normalised outcomes. Axis peak-to-peak spans in g; NULL uses acquisition metadata. Supply 16 for +/-8 g.
ref_axis_fsr_g	A supplied span must be a finite positive scalar. Missing reference range is retained as unavailable when no valid metadata exists; it is not needed to normalise test-device errors. Axis peak-to-peak spans in g; NULL uses acquisition metadata. Supply 16 for +/-8 g.
window_s	Duration in seconds of within-stage VM averaging bins used for the repeatability_error_cv diagnostic. It is separate from the pooled sample-level response fit and the trial-to-trial repeatability calculation.
epoch_time	Retained metadata setting end/center/start. In this report it does not change the epoch bins used for within-stage repeatability.
compute_entropy	Logical; calculate sample and approximate entropy using optional TSEntropies. Disabled or unavailable implementations return NA; this is a diagnostic, not an equivalence decision.
entropy_m	Embedding dimension for entropy calculations, passed as dim to SampEn/ApEn. Default two on the standardised VM series.
entropy_r	Entropy matching tolerance in units of the standardised VM series. NULL uses 0.2 times its sample SD. This is not an acceleration equivalence margin.
entropy_tau	Sample lag between embedded entropy coordinates, passed as lag to TSEntropies. Default one sample; its time duration depends on sampling rate.
verbose	Logical; print processing messages. Does not change calculations.
progress	Logical; display a text progress bar. Does not change calculations.
add_plot	Generate diagnostic ggplot objects.
inspect_alignment	Enable alignment diagnostics. These estimate lags without correcting timestamps or response metrics.
align_hz	Frequency in Hz of the uniformly resampled diagnostic signal used for lag estimation. It does not resample the data used in response/error calculations.
align_max_lag_s	Maximum lag magnitude in seconds searched by diagnostic cross-correlation; default 7200. Repetitive excitation can yield ambiguous peaks. The estimated lag is not applied as a timestamp correction.
align_by	Diagnostic channel: vm recombines all axes as vector magnitude; ax, ay or az selects one axis. This choice affects lag inspection only.
align_plot_stages_n	Retained compatibility argument; currently does not limit plotting or change calculations. It is not an active stage-selection control.

`compute_stage_xcorr`
: Logical; also estimate a lag separately for each matched stage when inspect_alignment is TRUE. Whole-trial lag diagnostics are separate; no lag is applied automatically.

`comp_plot_max_n`
: Maximum number of samples displayed in composite-response diagnostic plots, selected at evenly spaced indices. Fits and errors continue to use the full data.

`comp_plot_ylim`
: NULL uses the observed composite-trace plotting range; otherwise a numeric lower/upper g pair controls the displayed vertical interval. It does not exclude samples from fitting.

Details

Test vector magnitude is regressed on reference vector magnitude by ordinary least squares. Unequal lengths are matched by nearest positions on an evenly spaced sample-index grid, not by timestamp alignment or continuous interpolation. Stage matching is separate from pooled matching: pooled reference concatenation is length-matched once after concatenation. The caller must select synchronous, suitable stages and assess timing sensitivity. The pairing policy and measurement-range sources are reported in meta. The stage VM integral non-linearity denominator is $\sqrt{3}F$; the composite denominator is $\sqrt{3}F/2$, where F is test-axis peak-to-peak span. This implemented VM normalising span is not the nonnegative VM output range. Signed VM bias is the stage mean test-minus-reference difference; take its absolute value for absolute VM bias and convert slope minus one to percent for scale-factor error. Whole-stage VM FFT diagnostics are distinct from five-second pooled-triaxial spectral feature estimators. `dead_band` and `activation_threshold` are unestimated NA placeholders. Dynamic frequency diagnostics remain available; frequency-domain feature extraction and inter-device equivalence analyses are provided by WAM (https://github.com/miltheo/wam).

Value

A list: `stages` (one row per paired stage), `overall` (pooled response/composite error), `plots` (ggplot objects, possibly empty), and `meta` (ranges, windows, stage names and alignment diagnostics). Slope and scale-factor error are dimensionless; intercept/bias/SD are in g. VM integral non-linearity (retained field `linearity_error_vs_reference`) and composite error are percentages. Within-stage epoch-mean VM CV (retained field `repeatability_error_cv`) and other CV/utilisation fields are fractions. Trial-to-trial repeatability requires repeated assessments and separate aggregation.

Implementation

Implemented by `dynamic_metrology_report` in `R/m_dynamic.R`. See implementation-guide for the framework and function map.

Returned stage fields

Let F denote the configured test-axis peak-to-peak span, v the test VM in g, and r the length-matched reference VM. Fields use the existing stage-level calculations.

stage, rpm Matched stage name and its parsed numerical RPM label.

fs_hz_test, fs_hz_ref, n Timestamp-derived sampling frequencies in Hz and the number of test samples.

vm_mean, vm_standard_deviation, vm_mean_amplitude_deviation Mean, sample SD and mean absolute deviation from the stage mean of v, all in g.

vm_coefficient_of_variation Sample SD divided by mean VM, a fraction; undefined for zero mean.

signal_power_vm Mean of squared sample VM, in g squared, including its mean component. This is a signal mean square, not physical mechanical power.

dominant_frequency Frequency label in Hz of the maximum retained Fourier magnitude of demeaned stage VM. The retained frequency grid spans zero to half the derived rate across floor(N/2) entries, where N is the finite demeaned series length. This whole-stage diagnostic is separate from five-second pooled-triaxial feature frequency estimates.

dominant_frequency_magnitude Maximum unnormalised absolute discrete Fourier coefficient of demeaned stage VM over the retained half spectrum. It scales with record length; no division by sample count, single-sided amplitude scaling or spectral-density normalisation is applied. It is not a calibrated sinusoidal amplitude or a power spectral density.

sample_entropy, approximate_entropy Dimensionless SampEn/ApEn of standardised VM with the supplied parameters; NA when disabled, unavailable or unsuccessful.

vm_bias_vs_reference Mean signed v minus r, in g; not mean absolute paired error.

response_slope, response_intercept, scale_factor_error OLS test-on-reference slope (dimensionless), intercept (g) and slope minus one (fraction).

linearity_error_vs_reference 100 times the maximum absolute fitted residual divided by sqrt(3) F, a percentage.

snr_time_domain Mean VM divided by sample SD of VM. A linear dimensionless ratio, not decibels or a separately measured signal/noise decomposition.

snr_frequency_domain Squared dominant Fourier magnitude divided by the sum of squared remaining retained magnitudes. A linear dimensionless ratio; zero remainder gives NA.

repeatability_error_cv SD divided by mean of within-stage window-mean VM, a fraction. It is not trial-to-trial repeatability.

full_scale_utilisation_axis Maximum absolute scalar axis observation divided by F, a fraction. The denominator is the full peak-to-peak span, not the positive axis limit F/2.

full_scale_utilisation_vm Maximum VM divided by sqrt(3) F, a fraction. It uses the implemented normalising span, not the maximum nonnegative VM supported by a symmetric three-axis sensor.

dead_band, activation_threshold Unestimated NA placeholders.

alignment_lag_s Optional stage cross-correlation lag in seconds; not applied to sample timestamps or errors.

The one-row `overall` table contains `composite_error` (100 times the pooled maximum absolute fitted residual divided by sqrt(3) F/2), `response_slope`, `response_intercept` and `scale_factor_error`. Pooled fitting uses concatenated matched stages. See Details for its distinct reference length matching. The diagnostic plots and metadata accompany these tables; they do not alter the response fits.

See Also

`metronautica-package`

`feature-comparison` *Compare feature pipelines over one dynamic recording*

Description

Compare epoch summaries from the same decoded recording with common time bounds and explicit processing settings.

Details

ENMO, mean gravity-subtracted SVM and band-pass SVM are overlaid on a common mg axis. MAD uses a separate scale in the same figure because its range is smaller in this recording. Values are not amplitude-normalised. The band-pass feature applies its own 0.69–10 Hz axis filter; do not prefilter the recording to request it. Five-second epochs use the same start label for these sample-level features. No resampling or additional VM calibration is requested. Shaded intervals indicate the ten six-minute excitation stages; the intervening adjustments remain visible. This is a within-recording comparison, not evidence of device equivalence or behavioural validity.

Value

The example creates `comparison`, containing long/wide feature tables and resolved settings, and `comparison_plot`, a ggplot object. Plot grouping changes only the display; retain the original result and settings for subsequent analysis.

Inputs

Obtain the LEAP dynamic attachment and catalogue from the canonical WAM sample guide, https://github.com/miltheo/wam/blob/main/samples/README.md, then configure the `metronautica.sample_dir` option with the local sample directory. This comparison does not require the reference recording.

See Also

- `compute_accel_summaries`
- `accel_metric_enmo`
- `accel_metric_mad`
- `accel_metric_svmg`
- `accel_metric_svmbp`

Examples

```
## Not run:
library(metronautica)
library(ggplot2)

path <- m_load_sample_data(device = "LEAP", type = "dynamic",
    include_reference = FALSE)
info <- attr(path, "sample_info")
recording <- parse_raw_auto(path, return = "bundle",
    time_shift_s = info$time_shift_s,
    full_scale_range_g = info$full_scale_range_g)
```

```
comparison <- compute_accel_summaries(recording,
    measures = c("ENMO", "SVMGMEAN", "SVMBP_069_10", "MAD"),
    epoch_s = 5, epoch_anchor = "start", quiet = TRUE)
comparison$settings$metric_units

plot_data <- comparison$data_long
main_panel <- "ENMO, mean gravity-subtracted SVM and band-pass SVM"
plot_data$panel <- factor(ifelse(plot_data$measure == "MAD",
    "MAD", main_panel), levels = c(main_panel, "MAD"))
start <- as.POSIXct(info$trial_start_utc, tz = "UTC")
stages <- data.frame(start = start + (0:9) * 8 * 60,
    end = start + (0:9) * 8 * 60 + 6 * 60, rpm = seq(50, 500, 50))
comparison_plot <- ggplot(plot_data, aes(time, value, colour = measure, linetype = measure)) +
    geom_rect(data = stages, aes(xmin = start, xmax = end,
        ymin = -Inf, ymax = Inf), inherit.aes = FALSE,
        fill = "grey92", colour = NA) +
    geom_line(linewidth = 0.5) +
    geom_text(data = stages, aes(x = start + 3 * 60, y = Inf,
        label = paste(rpm, "RPM")), inherit.aes = FALSE,
        vjust = 1.4, size = 2.7) +
    facet_wrap(~panel, ncol = 1, scales = "free_y") +
    scale_colour_manual(values = c(ENMO = "#0072B2", SVMGMEAN = "#D55E00",
        SVMBP_069_10 = "#009E73", MAD = "#CC79A7"),
        breaks = c("ENMO", "SVMGMEAN", "SVMBP_069_10", "MAD"),
        labels = c("ENMO", "Mean gravity-subtracted SVM",
            "Band-pass SVM (0.69-10 Hz)", "MAD")) +
    guides(colour = guide_legend(override.aes = list(
        linetype = c("solid", "22", "solid", "solid")))) +
    scale_linetype_manual(values = c(ENMO = "solid", SVMGMEAN = "22",
        SVMBP_069_10 = "solid", MAD = "solid"), guide = "none") +
    scale_x_datetime(date_labels = "%H:%M", date_breaks = "10 min",
        timezone = "UTC", expand = expansion(mult = c(0, 0.01))) +
    scale_y_continuous(expand = expansion(mult = c(0.03, 0.18))) +
    labs(x = "Time (UTC)", y = expression("Epoch mean (m"*italic(g)*")"),
        colour = NULL,
        title = "LEAP: feature pipelines over one dynamic trial",
        subtitle = paste("Five-second epochs; native decoded signal;",
            "no additional VM calibration")) +
    theme_bw(base_size = 11) +
    theme(legend.position = "bottom", strip.text.y = element_text(angle = 0),
        strip.background = element_rect(fill = "white"))
comparison_plot

## End(Not run)
```

`filter_accel_tbl` *Filter axes with a compensated Butterworth filter*

Description

Apply forward-backward Butterworth filtering to each acceleration axis and record processing provenance.

Usage

```
filter_accel_tbl(x, cutoff, forder = 4, ftype = "low")
```

Arguments

x	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.
cutoff	One cutoff for low/high pass, or increasing lower and upper cutoffs for pass/stop bands, in Hz.
forder	Positive integer Butterworth order before forward-backward application.
ftype	Filter type accepted by `signal::butter`: `"low"`, `"high"`, `"pass"`, `"stop"`.

Details

The implemented cutoff adjustment is $f_c/(\sqrt{2}-1)^{1/(2n)}$, followed by normalisation to Nyquist and `signal::filtfilt()`. All adjusted cutoffs must lie strictly between zero and Nyquist. This helper applies the cutoff adjustment for the selected filter type; do not assume identical effective edges across filter types. The feature-suite band-pass filter is a separate implementation. The input must contain a finite positive acquisition$fs_derived_hz in its metadata; this helper does not substitute a nominal rate or infer a new rate. Regular sampling and an adequate contiguous, finite record are caller responsibilities; this function does not repair gaps.

Value

The same input container type with filtered axes, unchanged timestamps, appended processing history and invalidated inherited QC. Table and bundle metadata remain synchronised.

Implementation

Implemented by `filter_accel_tbl` in `R/m_filter.R`. See `implementation-guide` for the framework and function map.

Stage bundles

For a parser bundle, the filtered recording is returned in `data`. For a dynamic stage bundle, only `full` is filtered; `stages` and `schedule` are retained unchanged. They therefore do not become filtered stage inputs. Filter the acceleration table or parser bundle first, then call `clip_dynamic_stages` on the filtered result. The returned container is not a recursive transformation of every stored signal.

See Also

`metronautica-package`

`getting-started` *Start with an offline acceleration workflow*

Description

Construct a small common-format recording, inspect diagnostics, calculate features and retain their settings without external data or optional packages.

Details

Install metronautica and its Imports dependencies, then load it with `library(metronautica)`. The example below runs offline. The waveform is synthetic and demonstrates the interface; it is not a validation recording or a stationary noise measurement.

For a native recording, replace the constructor with `parse_raw_auto` and inspect the returned metadata. Parsing attaches QC by default; the constructor requires explicit `attach_qc`. Optional native decoders depend on format. Interactive signal plots require plotly; calculations below do not.

For stationary metrology, select a stationary window with `static_clip`, then use `static_metrology_from_tbl` or its direct outcome helpers. For stepped excitation, apply any intended filter to the recording before `clip_dynamic_stages`, construct test/reference stages and call `dynamic_metrology_report`. `native-recording-examples` supplies stationary and orbital-shaker examples with external recordings. The `implementation-guide` maps these stages to source functions and measurement outcomes.

See Also

`common-data-format`, `compute_accel_summaries`, `native-recording-examples`

Examples

```
library(metronautica)
fs <- 100
seconds <- (0:1999) / fs
meta <- list(acquisition = list(fs_nominal_hz = fs,
    fs_derived_hz = fs, full_scale_range_g = 16))
x <- as_accel_tbl(as.POSIXct("2026-01-01", tz = "UTC") + seconds,
    .2 * sin(2*pi*2*seconds), .2 * cos(2*pi*2*seconds),
    rep(1, length(seconds)), meta)
x <- attach_qc(x, scope = "synthetic 20-second recording")
accel_meta(x)$processing$qc[c("n_samples", "median_dt_ms")]

features <- compute_accel_summaries(x, measures = c("ENMO", "MAD"),
    epoch_s = 5, fs_hz = fs, quiet = TRUE)
head(features$data_wide)
features$settings$metric_units
stopifnot(nrow(features$data_wide) == 4)

# AI uses one-second bin-start labels rather than sample-level labels.
# Explicit zero disables noise subtraction only for this interface demonstration.
mixed <- compute_accel_summaries(x, measures = c("ENMO", "AI"),
    epoch_s = 5, fs_hz = fs, ai_noise_var = 0, quiet = TRUE)
table(mixed$data_long$measure)  # Four epochs per feature
nrow(mixed$data_wide)          # Eight rows: different timestamp labels
stopifnot(nrow(mixed$data_wide) == 8)
```

`implementation-guide` *Framework and function map*

Description

Connect the public functions, source files and implemented measurement outcomes. Function help documents the resolved parameters; this guide identifies the measurement represented by each stage.

Common data format and signal diagnostics

R/m_parsers.R implements decoding to POSIXct time and triaxial acceleration in g. Stored device coefficients may already be applied during decoding; native does not mean unscaled integer counts. Consult parse_raw_auto for the supported format and validated LEAP scope.

R/m_qc.R implements five post-parse signal diagnostic groups. qc_accel_tbl explains exact numerators, denominators, sequence eligibility and the near-limit threshold. These are operational screening rules, not a sensor acceptance standard. attach_qc stores the results; invalidate_qc removes diagnostics that no longer describe a modified signal.

Static characterisation

R/m_static.R implements static characterisation outcomes. static_1hz_means supplies one-second means; static_drift fits signed g/h slopes; static_offset_vm_5min estimates the specified initial VM offset relative to 1 g. static_axis_noise_metrics separates raw sample SD from the residual SD after linear detrending. The static noise floor is the maximum axis-specific detrended residual SD, not the Activity Index stationary variance.

static_allan uses allan_nonoverlap for non-overlapping cluster means. Its returned axis minima and evaluated averaging times are separate from the maximum across individual axis minima. Supplying a stationary window and assessing environmental stability remain experimental responsibilities.

Dynamic characterisation and calibration

R/m_dynamic.R implements explicit stage selection and VM response calculations. m_dynamic_response fits test VM on reference VM and returns signed bias, response slope/intercept and residual-based errors. The stage normalising span is sqrt(3) times the test-axis peak-to-peak span; the composite denominator is half that span. These operational definitions do not imply that VM occupies the symmetric axis measurement interval.

dynamic_metrology_report reports within-stage epoch-mean VM CV as a fraction. It is not trial-to-trial repeatability CV. Trial aggregation, activation threshold, deadband estimation, inter-device equivalence and Holm-adjusted regional tests are provided by WAM (https://github.com/miltheo/wam). The report's activation/deadband columns are unestimated placeholders.

Reference length matching remains nearest selection on an evenly spaced sample-index grid. Stage-wise and pooled matching are separate. Neither that operation nor diagnostic cross-correlation constitutes timestamp correction. dm_fit_vm_calibration estimates a VM mapping; dm_apply_vm_calibration_tbl applies that mapping and invalidates inherited QC. Independent validation is required to assess benefit outside the fitting recording.

Application-specific features

R/m_accel_summary_suite.R implements time-domain feature definitions and configurable alternatives. The exact operation and unit are given in each accel_metric_* help page. compute_accel_summaries combines requested features and returns resolved pipeline settings.

Epoch duration, sampling treatment, axis filtering, noise correction, calibration state and aggregation are part of a feature pipeline. ENMO truncates negative VM-minus-1-g values per sample; SVMgs variants use absolute differences. MAD is centred within the epoch. ROCAM is a sample-to-sample acceleration-change outcome with no elapsed-time division. The package does not infer physical activity, sleep, energy expenditure or behavioural validity from these features. See feature-comparison for a combined plot over one dynamic recording.

Standalone activity counts

`accel_metric_agcounts` in R/m_agcounts.R implements the acknowledged published count algorithm and explicit custom-filter alternatives. It operates as a standalone function, outside the summary selector. Report input precision, gap and boundary policies as well as filter/epoch settings; processing flags are not a complete uncertainty budget.

Error and uncertainty reporting

VIM terminology and the cited IEEE sensor terminology/performance standards are used to distinguish measurement quantities and properties. The package returns operational estimates; it does not establish traceability or a complete uncertainty budget by itself. Report the reference, tested range, signal state, sampling/epoch/filter settings, timing diagnostics and units. Residual errors, between-unit variability and uncertainty of a downstream estimate are distinct quantities. See `measurement-definitions` for the calculation definitions.

Input and return contracts

Raw-signal helpers accept the common four-column acceleration table or data/full bundles; extracting a bundle does not imply selecting, sorting or repairing samples. The table's metadata attribute is authoritative, with top-level meta as a fallback. Calculation helpers generally return a derived result, rather than the original container. Container behaviour depends on the operation: static_clip always returns an acceleration table; clip_dynamic_stages creates a new stage bundle; clip_bundle, QC helpers, filtering and calibration preserve their documented table/bundle form. For a dynamic stage bundle, filter_accel_tbl and clip_bundle modify only the selected full recording, leaving existing stages unchanged. Filter or clip the recording before constructing the stage bundle; do not use the unchanged stages as filtered or newly clipped data. Calibration application explicitly processes the stored stages as well as the full recording.

Derived-input helpers are deliberately different: static_drift and static_offset_vm_5min require one-second means; Allan-minimum/plot functions require an Allan table; allan_nonoverlap requires a numeric axis series; dynamic reporting and fitting require corresponding named stage bundles. m_dynamic_response also accepts numeric VM vectors. Parsers/sample loaders take paths or sample selectors, not acceleration tables. These distinctions follow the quantity each function calculates rather than imposing a common shell API.

References

Joint Committee for Guides in Metrology (2012). International vocabulary of metrology: Basic and general concepts and associated terms, JCGM 200:2012. doi:10.59161/JCGM2002012.

IEEE (2018). IEEE Std 2700-2017, Standard for Sensor Performance Parameter Definitions. doi:10.1109/IEEESTD.2018.8277147.

IEEE (2019). IEEE Std 1293-2018, Standard Specification Format Guide and Test Procedure for Linear Single-Axis, Nongyroscopic Accelerometers. doi:10.1109/IEEESTD.2019.8653544.

Bai et al. (2016). An activity index for raw accelerometry data and its comparison with other activity metrics. doi:10.1371/journal.pone.0160644.

Neishabouri et al. (2022). Quantification of acceleration as activity counts in ActiGraph wearable. doi:10.1038/s4159802216003x.

See Also

`metronautica-package`, `measurement-definitions`, `native-recording-examples`

invalidate_qc *Remove diagnostics inherited from an earlier signal state*

Description

Remove diagnostics inherited from an earlier signal state.

Usage

```
invalidate_qc(x)
```

Arguments

x	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.

Value

The same input container with inherited QC removed; acceleration samples are unchanged.

Implementation

Implemented by `invalidate_qc` in `R/m_qc.R`. See `implementation-guide` for the framework and function map.

See Also

`metronautica-package`

measurement-definitions *Calculation definitions and interpretation*

Description

Units, normalisation and implemented arithmetic for the public metrology functions.

Details

Vector magnitude is $v = \sqrt{a_x^2 + a_y^2 + a_z^2}$ in g.

Signed 1-g VM offset bias Initial one-second mean VM minus 1 g, averaged over elapsed time up to and including five minutes. This is a VM offset relative to gravity, not a complete axis calibration.

Signed VM drift rate Ordinary least-squares slope against elapsed hours, g/h. Extrapolation beyond the stationary recording requires an explicitly stated model.

Static noise floor Maximum axis-specific residual SD after linear detrending over the selected stationary window; reporting in mg requires a factor of 1000. The first-five-minute maximum-axis SD is raw, not detrended. Raw sample SD uses denominator n-1; linearly detrended residual SD uses n-2. Static CV is 100 times SD divided by absolute mean, with a near-zero guard.

Allan deviation For non-overlapping cluster means $\bar{x}_j$, $\sigma_A = \sqrt{\text{mean}[(\bar{x}_{j+1} - \bar{x}_j)^2]/2}$. Cluster duration is in seconds and deviation is in g.

Dynamic response and signed VM bias OLS of test VM on length-matched reference VM. Slope is dimensionless; intercept and mean signed test-minus-reference bias are in g. Scale-factor error is slope minus one; multiply it by 100 for percentage reporting. Absolute VM bias is the absolute value of the stage-mean signed difference, not the mean absolute paired difference.

VM integral non-linearity 100 times the maximum absolute fitted residual divided by $\sqrt{3}F$, where F is the test-axis peak-to-peak span in g.

Trial-level composite error 100 times the maximum absolute residual from the pooled response fit divided by $\sqrt{3}F/2$. This denominator differs from stage VM integral non-linearity by a factor of two. The implemented normalising span is not the nonnegative VM output range.

Within-stage epoch-mean VM CV SD of within-stage epoch mean VM divided by their mean. The retained field `repeatability_error_cv` is a fraction. It is distinct from trial-to-trial VM repeatability CV, defined as 100 times SD divided by mean of stage-mean VM across repeated trials for the same unit, stage and pipeline.

Maximum-axis Allan minimum Maximum across axes of their individual minimum Allan deviations over the evaluated averaging times; reporting in micro-g requires a factor of one million. This is distinct from the axis-specific curves.

ENMO Epoch mean of $\max(v-1, 0)$. Negative differences are truncated to zero per sample before averaging.

MAD Epoch mean of $|v - \bar{v}|$, with $\bar{v}$ defined within the epoch. ENMO and MAD are multiplied by 1000 for default mg output.

Index-based dynamic reference matching is not timestamp correction. No equivalence margin or universal fit-for-purpose decision is embedded in these functions. Report the reference, tested range, rate, filtering, epoch settings, signal calibration state, error normalisation and relevant timing diagnostics with every application.

See Also

- `metronautica-package`.
- `static_metrology_from_tbl`.
- `dynamic_metrology_report`.
- `compute_accel_summaries`.

`m_dynamic_response` *Compute vector-magnitude response and reference-dependent error*

Description

Compute vector-magnitude response and reference-dependent error.

Usage

```
m_dynamic_response(test, reference, axis_fsr_g)
```

Arguments

test	Test-device vector magnitude in g, or an acceleration table/bundle converted to vector magnitude.
reference	Reference vector magnitude in g, or an acceleration table/bundle. Nearest evenly spaced reference indices match the test length.
axis_fsr_g	Required finite positive scalar. Configured single-axis peak-to-peak span in g. The VM normalising span is sqrt(3) times this value.

Details

The least-squares test-VM-on-reference-VM regression and nearest-index length matching are the same calculations used by dynamic_metrology_report. The signed bias is the mean test-minus-reference VM. Scale-factor error is slope minus one, dimensionless. Integral non-linearity uses the maximum absolute residual divided by the VM normalising span, times 100. The composite-error field uses half that span; it represents trial composite error only when the supplied vectors contain the complete pooled trial assessment. It is not a measurement-uncertainty budget. This function applies no new calibration or timestamp correction.

Value

A list with vm_bias_vs_reference (signed mean difference, g), response_slope (dimensionless), response_intercept (g), scale_factor_error (slope minus one), linearity_error_vs_reference and composite_error (percentages using the full and half normalising spans respectively), maximum_absolute_residual_g, and predicted_vm_g (fitted test VM values in g).

Implementation

Implemented by `m_dynamic_response` in `R/m_dynamic.R`. See implementation-guide for the framework and function map.

See Also

metronautica-package

Examples

```
reference_vm <- seq(.1, 2, length.out = 100)
test_vm <- .02 + 1.1 * reference_vm
response <- m_dynamic_response(test_vm, reference_vm, axis_fsr_g = 16)
response[c("response_slope", "response_intercept", "composite_error")]
```

`m_load_sample_data` *Retrieve a native recording example into a project*

Description

Copy a local native example or explicitly download it from a configured fixed WAM release-asset directory, verifying compressed and restored checksums. No measurement values are changed.

Usage

```
m_load_sample_data(device = "LEAP", type = "static", dest_dir =
    file.path(getwd(), "metronautica-samples"), overwrite = FALSE,
    sample_dir = NULL, include_reference = TRUE,
    base_url = getOption("metronautica.sample_base_url", NULL))
```

Arguments

`device`	One of LEAP, AX3, SENS or GENEActiv; matching is case-insensitive.
`type`	Either static or dynamic.
`dest_dir`	Directory in which the native recording and its provenance sidecar are created when this function is explicitly called.
`overwrite`	Allow replacement of a different existing target file, only after validation. An identical file is reused without accessing its source.
`sample_dir`	Directory containing sample_index.csv and compressed samples. Defaults to the option `metronautica.sample_dir`, if set. Without a local catalogue, the installed package catalogue is used.
`include_reference`	Retrieve the primary HBK reference with dynamic samples. Ignored for static samples.
`base_url`	Fixed HTTPS WAM release-asset directory, used only when a needed compressed sample is unavailable locally. Defaults to `metronautica.sample_base_url`. No public URL is preconfigured. A `file:///` directory also supports offline retrieval.

Details

Stationary and orbital-shaker recording examples are distributed separately through WAM (https://github.com/miltheo/wam), not installed with this package. For local source checkouts, point sample_dir at WAM's samples directory. After publication, configure base_url with the fixed sample-data release-asset directory. The packaged catalogue fixes filenames and SHA-256 checksums. Installation, package loading and ordinary calculations do not require WAM or network access. If a sample cannot be obtained, an informative error explains local and remote configuration. Existing identical restored files are reused offline; existing different files are protected unless overwrite is TRUE. Downloads and decompression use temporary files and validation precedes replacement.

Recording intervals, native-format boundary handling, provenance and distribution terms are maintained in the WAM sample guide and catalogue: https://github.com/miltheo/wam/blob/main/samples/README.md. Apply each returned file's own timestamp shift and acquisition settings when parsing. This function does not parse, filter or calibrate data.

Value

The native file path, with a sample_info attribute containing source identity, analysis bounds, units and timestamp shift. A JSON sidecar contains the same information. With a dynamic reference, the path also has reference_file and reference_info attributes.

Implementation

Implemented by `m_load_sample_data` in `R/m_sample_data.R`. See `implementation-guide` for the framework and function map.

See Also

`native-recording-examples`, `parse_raw_auto`

Examples

```
## Not run:
# Local sibling checkout; adjust for the current project.
options(metronautica.sample_dir =
    "../wam/samples")
path <- m_load_sample_data(device = "LEAP", type = "dynamic",
                           dest_dir = tempdir())
info <- attr(path, "sample_info")
x <- parse_raw_auto(path, return = "bundle",
                    time_shift_s = info$time_shift_s,
                    full_scale_range_g = info$full_scale_range_g)
accel_meta(x)$processing$qc$ruleset

## End(Not run)
```

---

native-recording-examples

Worked examples using stationary and orbital-shaker recordings

---

Description

Parse native recordings, inspect primary-five diagnostics, compute time-domain features and characterise static and dynamic signals using direct functions.

Details

Recording contents, selection boundaries, provenance and distribution terms are documented in the canonical WAM sample guide: https://github.com/miltheo/wam/blob/main/samples/README.md. The examples below demonstrate the loader and direct processing functions; they do not establish between-device equivalence or behavioural validity.

Input and processing

The sample loader copies native files to the project, checks their SHA-256 digests and writes provenance sidecars. Dynamic examples include the primary HBK reference by default. The returned path carries sample_info, reference_file and reference_info attributes. Apply each file's documented timestamp shift explicitly when parsing. Parsing runs signal diagnostics by default without repairing or removing observations. Tables and parser bundles are accepted by the calculation functions. Optional GENEAread is needed for GENEActiv; optional plotly is needed for the interactive plots.

Interpreting the examples

The static window is selected explicitly before drift, noise and Allan-deviation calculations. Full static parsing can take several minutes and substantial memory. The ten-stage example supplies one-minute trimming at each stage end; the native recording remains unchanged. Reference matching uses the retained nearest sample-index method, separate from timestamp-shift and alignment diagnostics. One unit and one trial illustrate calculations, not between-unit inference, trial-to-trial repeatability or device equivalence. Consult the individual help pages for normalising spans and units.

Sample distribution

Distribution and provenance are specified by the canonical WAM sample guide linked above. Configure `options(metronautica.sample_dir = path)` for local samples, or `metronautica.sample_base_url` with the fixed HTTPS release-asset directory after publication. Existing identical restored files are reused offline. No data are downloaded during package installation or loading.

See Also

- m_load_sample_data
- parse_raw_auto
- static_metrology_from_tbl
- dynamic_metrology_report
- m_dynamic_response
- measurement-definitions

Examples

```
## Not run:
# Configure metronautica.sample_dir with the catalogue and recording attachments.
library(metronautica)

# Static characterisation
path <- m_load_sample_data(device = "LEAP", type = "static")
info <- attr(path, "sample_info")
stationary <- parse_raw_auto(path, return = "bundle",
    time_shift_s = info$time_shift_s,
    full_scale_range_g = info$full_scale_range_g)
static_result <- static_metrology_from_tbl(stationary,
    start_utc = info$analysis_start_utc, duration_h = 24, progress = FALSE)
static_result$summary
static_result$drift_plot
static_result$allan_plot

# Dynamic characterisation
path <- m_load_sample_data(device = "LEAP", type = "dynamic")
info <- attr(path, "sample_info")
ref_info <- attr(path, "reference_info")
test <- parse_raw_auto(path, return = "bundle",
    time_shift_s = info$time_shift_s,
    full_scale_range_g = info$full_scale_range_g)
reference <- parse_raw_auto(attr(path, "reference_file"), return = "bundle",
    time_shift_s = ref_info$time_shift_s,
```

```
    full_scale_range_g = ref_info$full_scale_range_g)
test_stages <- clip_dynamic_stages(test, start_utc = info$trial_start_utc,
    n_stages = 10L, stage_min = 6, buffer_min = 2, rpm_start = 50, rpm_step = 50,
    trim_start_min = 1, trim_end_min = 1)
ref_stages <- clip_dynamic_stages(reference, start_utc = ref_info$trial_start_utc,
    n_stages = 10L, stage_min = 6, buffer_min = 2, rpm_start = 50, rpm_step = 50,
    trim_start_min = 1, trim_end_min = 1)
dynamic_result <- dynamic_metrology_report(test_stages, ref_stages,
    test_axis_fsr_g = info$full_scale_range_g,
    ref_axis_fsr_g = ref_info$full_scale_range_g,
    compute_entropy = FALSE, inspect_alignment = FALSE, add_plot = TRUE,
    progress = FALSE)
dynamic_result$stages
dynamic_result$overall
dynamic_result$plots$vm_mean_vs_rpm
dynamic_result$plots$linearity_vs_rpm

# Feature extraction
features <- compute_accel_summaries(test, measures = c("ENMO", "MAD"), epoch_s = 5)
head(features$data_wide)
features$settings$metric_units
if (requireNamespace("plotly", quietly = TRUE)) {
    plot_processed_metric(features, "ENMO")
}

## End(Not run)
```

`parse_raw_auto` *Decode a supported accelerometer file*

Description

Detect the device format, decode acceleration in g and retain timestamp, parser and calibration provenance.

Usage

```
parse_raw_auto(path, return = c("data", "bundle"), verbose = FALSE,
    progress = FALSE, time_shift_s = 0, tz_parsed = "UTC",
    hash_sha256 = NULL, qc = TRUE, ...)
```

Arguments

`path`	Path to one existing device file.
`return`	Output form: `"data"` or `"bundle"`.
`verbose`	Logical; print processing messages. Does not change calculations.
`progress`	Logical; display a text progress bar. Does not change calculations.
`time_shift_s`	Additive timestamp shift in seconds. No clock-rate correction is inferred.
`tz_parsed`	Timezone for supported text timestamp decoding and Axivity CWA reading; output is tagged UTC. Binary formats may use device-defined time policies.
`hash_sha256`	Optional caller-supplied source digest. NULL computes it; a supplied digest is recorded, not independently verified.

qc	Run primary-five diagnostics after decoding (default TRUE). FALSE skips these diagnostics.
...	Named device-specific settings listed under Details. Only pass settings supported by the detected parser.

Details

Supported inputs are GENEActiv/SENS BIN, Axivity CWA/AX3 CSV, ActiGraph GT3X/LEAP CSV/LEAP AGSD and HBK G-Link CSV. GENEAread, GGIRread (CWA fallback) and read.gt3x are optional decoder dependencies. Unavailable dependencies or unsupported formats raise errors; packages are never installed during parsing. The AGSD calibration decoder is bounded to LEAP v2, dataVersion 3, firmware 2.1.0.106, 128 Hz and calibrationMethod 2. Unsupported configurations fail rather than being silently extrapolated. Device decoding coefficients do not establish an independently traceable calibration. Timezone assumptions and shifts must be reviewed before reference pairing. QC runs by default and is stored under meta$processing$qc with ruleset primary-five. A standalone source session must also load m_qc.R. The shifted-reciprocal LEAP conversion verified against paired device exports is the default; supported configuration checks are retained.

Value

An `accel_tbl`, or `list(data = accel_tbl, meta = list)`. Metadata records the source, acquisition settings, decoding/calibration operations and integrity diagnostics. The automatic parser identity is `"m_parsers"` in `meta$file$auto_parser` and `meta$processing$parser_library$name`; `meta$file$parser` identifies the device-specific decoder.

Implementation

Implemented by `parse_raw_auto` in `R/m_parsers.R`. See `implementation-guide` for the framework and function map.

Device-specific arguments through ...

full_scale_range_g Configured symmetric peak-to-peak axis span in g; 16 means -8 to +8 g. Use file metadata when available or supply the documented acquisition setting. It affects range diagnostics and later normalisation, not already decoded CSV acceleration values. Observed minimum/maximum values are not a substitute for sensor limits.

GENEActiv calibrate Logical, default TRUE. Forwarded to GENEAread::read.bin to control upstream coefficient application. This is device-file decoding/calibration, not the within-trial dynamic VM mapping. The returned provenance records whether upstream calibration was requested. mmap_load controls memory-mapped loading, default enabled on 64-bit R, without adding an analysis filter.

SENS scale_g_per_lsb Acceleration scale converting signed axis counts to g; default 0.0078125 from the supported vendor binary specification. A different value requires device-format justification and changes decoded acceleration. No per-file offset or temperature correction is inferred.

GT3X precision Default `"native"` preserves original integer-sample precision through file-scale conversion for supported modern log.bin containers, using the validated read.gt3x 1.2.0 interface. `"legacy_3dp"` selects the earlier upstream scaled/rounded output. Precision and native recording start are recorded in metadata. Native mode does not impute gaps or extract light; choose the explicit legacy mode for those upstream options. Older activity.bin layouts require legacy mode. No lost precision is inferred from CSV or rounded input.

GT3X imputeZeroes Logical, default FALSE: retain the supported upstream decoder's non-imputation route. Enabling it changes the decoded treatment of missing/idle data and must be reported. add_light defaults FALSE; cleanup controls upstream extraction-file cleanup.

Axivity CWA settings use_ggir_fallback defaults TRUE and permits the optional GGIRread path when the fast packed-AX3 layout is unsupported. interpolationType is passed to the fallback reader (default 1). Fast and fallback implementations and their scaling/time policies are retained in metadata; the caller should inspect the selected parser.

Sensor identifiers sensor_vendor and sensor_model optionally annotate the sensor hardware. They do not select conversion coefficients or establish calibration traceability. Only pass arguments accepted by the detected parser.

See Also

metronautica-package

`plot_accel_clip` *Plot triaxial acceleration and optional vector magnitude*

Description

Plot triaxial acceleration and optional vector magnitude.

Usage

```
plot_accel_clip(x, include_vm = FALSE, demean_vm = FALSE, title =
    NULL)
```

Arguments

`x`	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.
`include_vm`	Add sample-level vector magnitude to the triaxial plot.
`demean_vm`	For the optional VM trace, plot max(VM minus 1 g, 0) instead of VM.
`title`	Optional plot title.

Value

An interactive Plotly htmlwidget. Display thinning does not alter the input recording.

Implementation

Implemented by `plot_accel_clip` in `R/m_plotter.R`. See implementation-guide for the framework and function map.

See Also

metronautica-package

plot_processed_metric *Plot an epoch-level acceleration feature*

Description

Plot an epoch-level acceleration feature.

Usage

```
plot_processed_metric(result, metric, unit = NULL, title = metric)
```

Arguments

result	Summary result from compute_accel_summaries or its data_wide table.
metric	Name of the numeric column to plot from the processed summary table.
unit	Unit label displayed on the processed plot; does not rescale values. NULL uses result$settings$metric_units. Supply a label explicitly for a bare data_wide table or older outputs without this metadata; no mg unit is assumed.
title	Optional plot title.

Value

An interactive Plotly htmlwidget; no feature values are recalculated.

Implementation

Implemented by plot_processed_metric in R/m_plotter.R. See implementation-guide for the framework and function map.

See Also

metronautica-package

plot_static_allan *Plot axis-specific Allan deviation*

Description

Plot axis-specific Allan deviation.

Usage

```
plot_static_allan(allan_df, title_prefix = "Allan deviation")
```

Arguments

allan_df	Allan-deviation table with axis, tau and adev columns: axis label, evaluated averaging time in seconds, and deviation in g. Obtain it with static_allan.
title_prefix	Character prefix for diagnostic plot titles; does not change computed outcomes.

Details

Both axes use log10 coordinates with inward logarithmic tick marks. Axis minima and their averaging times are labelled from the supplied table; plotting does not recalculate Allan deviation.

Value

A ggplot object; supplied data are unchanged.

Implementation

Implemented by `plot_static_allan` in `R/m_static.R`. See `implementation-guide` for the framework and function map.

See Also

`metronautica-package`

`plot_static_drift_vm` *Plot signed vector-magnitude drift*

Description

Plot signed vector-magnitude drift.

Usage

```
plot_static_drift_vm(d1, drift_fit, title_prefix = "Static drift")
```

Arguments

`d1`	One-second summary table returned by static_1hz_means, with elapsed_h and axis/VM mean columns. elapsed_h is time since the first populated UTC-second bin, in hours. This is a derived table, not a raw parser bundle.
`drift_fit`	The fitted linear model, obtained as `static_drift(d1)$fit_vm`.
`title_prefix`	Character prefix for diagnostic plot titles; does not change computed outcomes.

Value

A ggplot object; supplied data are unchanged.

Implementation

Implemented by `plot_static_drift_vm` in `R/m_static.R`. See `implementation-guide` for the framework and function map.

See Also

`metronautica-package`

qc_accel_tbl *Quantify signal and timestamp integrity*

Description

Report integrity diagnostics without sorting, repairing or removing samples.

Usage

```
qc_accel_tbl(x)
```

Arguments

x	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. For this diagnostic function the table must have exactly time, ax_g, ay_g and az_g in that order. No samples are repaired by input conversion.

Details

The ruleset is named primary-five. The five groups below describe the decoded recording, without exclusion, sorting, interpolation or repair. Finite means neither missing, NaN nor infinite. Rounding tolerances are not introduced into exact equality tests.

1. **Data integrity** n_samples counts recorded rows. missing_time counts non-finite timestamps. duplicate_time counts repeated finite timestamps after their first occurrence. backward_time counts negative finite differences between adjacent timestamps. invalid_axis counts non-finite scalar axis values across X, Y and Z, so its denominator is three times the row count rather than the number of rows.
2. **Sample timing** Select only finite, strictly positive differences between adjacent recorded timestamps. median_dt_ms is their median, iqr_dt_ms is their interquartile range using R type-7 quartiles, and longest_dt_ms is their maximum, all in milliseconds. IQR requires at least two positive intervals. No positive intervals yields NA. These summaries expose spacing and gaps; they do not count missing samples or establish synchronisation with another recording.
3. **Axis-zero proportion** For each axis, zero_x_n/zero_y_n/zero_z_n count exact zero values. finite_x_n/finite_y_n/finite_z_n count finite values independently. Each zero percentage is 100 times its axis zero count divided by its finite count; no finite values gives NA. A single zero axis can be expected at some stationary orientations and is not an automatic failure.
4. **Unusual cross-axis patterns** all_zero_samples counts rows with X=Y=Z=0; equal_nonzero_samples counts rows with X=Y=Z and X not zero. Pattern counts require finite XYZ values. Each sequence flag is TRUE if its pattern occurs in at least two adjacent eligible rows: both timestamps and all axes must be finite and the timestamp step must be strictly positive. An invalid row, repeated timestamp or reversal breaks the sequence. No gap-duration limit is imposed. Equal non-zero values may differ between adjacent rows. These sample-level rules adapt the NHANES pattern concepts; they do not reproduce its minute-level algorithm or imply every flagged sample is defective.
5. **Near-limit values** limit_g is half the configured symmetric peak-to-peak axis span from acquisition$full_scale_range_g. near_limit_fraction is the fixed screening fraction 0.98. Pool finite scalar observations from all three axes and count those satisfying abs(a) >= 0.98*limit_g as

near_limit_n. finite_axis_n is the pooled finite count, and near_limit_pct is 100 times their ratio. Missing/invalid range metadata yields an undefined limit/count and percentage. This is a proximity-to-range screen, not proof of clipping or a calibrated saturation threshold.

The diagnostic thresholds, numerators and denominators are retained with the output so they can be reported. The 0.98 near-limit fraction and pooled-axis denominator are screening conventions; VIM measurement-span terminology does not prescribe that fraction. Stored QC describes only the signal and interval on which it was calculated. Recompute after processing changes or a new clip.

Value

A named list of scalar diagnostics. Counts are sample counts except `invalid_axis`, `near_limit_n` and `finite_axis_n`, which count axis observations. Percentages are on the 0–100 scale.

Implementation

Implemented by `qc_accel_tbl` in `R/m_qc.R`. See `implementation-guide` for the framework and function map.

See Also

`metronautica-package`

Examples

```
t0 <- as.POSIXct("2026-01-01 00:00:00", tz = "UTC")
fs <- 100
s <- (0:5999) / fs
meta <- list(acquisition = list(fs_derived_hz = fs,
    fs_nominal_hz = fs, full_scale_range_g = 16))
x <- as_accel_tbl(t0 + s, 0.1 * sin(2 * pi * 2 * s),
    0.1 * cos(2 * pi * 2 * s), 1 + 0.1 * sin(2 * pi * s), meta)
q <- qc_accel_tbl(x)
q[c("n_samples", "median_dt_ms", "near_limit_pct")]
```

`static_1hz_means` *Compute one-second triaxial and vector-magnitude means*

Description

Compute one-second triaxial and vector-magnitude means.

Usage

```
static_1hz_means(x)
```

Arguments

`x`	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.

Value

A tibble with sec (UTC second-bin start), ax_mean, ay_mean, az_mean and vm_mean (one-second means in g), and elapsed_h (hours since the first populated bin).

Method

Bin timestamps by floor of Unix seconds. Within each populated second, average each axis and sample vector magnitude, omitting missing values. Empty UTC seconds are not filled. elapsed_h starts at the first populated bin. This is the input to the static drift and initial VM-offset calculations.

Implementation

Implemented by `static_1hz_means` in `R/m_static.R`. See `implementation-guide` for the framework and function map.

See Also

`metronautica-package`

`static_allan` *Compute axis-specific non-overlapping Allan deviation*

Description

Compute axis-specific non-overlapping Allan deviation.

Usage

```
static_allan(x)
```

Arguments

x	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.

Value

A tibble containing tau, adev in g and axis identifiers.

Method

For each axis, evaluate 60 log-spaced requested averaging times from 0.1 to 10000 seconds through allan_nonoverlap. Sampling frequency is estimated from positive timestamp intervals. The calculation assumes suitable uniformly sampled stationary data; it does not repair gaps or resample. The minimum four-cluster requirement limits supported long averaging times.

Implementation

Implemented by `static_allan` in `R/m_static.R`. See `implementation-guide` for the framework and function map.

See Also

metronautica-package

`static_allan_minima_wide` *Summarise axis-specific Allan minima*

Description

Summarise axis-specific Allan minima.

Usage

```
static_allan_minima_wide(allan_df)
```

Arguments

`allan_df`	Allan-deviation table with axis, tau and adev columns: axis label, evaluated averaging time in seconds, and deviation in g. Obtain it with static_allan.

Value

A one-row tibble with x_allan_min_g, y_allan_min_g and z_allan_min_g (axis minima in g), plus x_cluster_time_s, y_cluster_time_s and z_cluster_time_s (the corresponding evaluated averaging times in seconds).

Method

Find the smallest evaluated deviation separately for X, Y and Z and retain its averaging time. A maximum-axis Allan minimum is obtained separately by taking the maximum of these three axis minima.

Implementation

Implemented by `static_allan_minima_wide` in `R/m_static.R`. See implementation-guide for the framework and function map.

See Also

metronautica-package

`static_axis_noise_metrics` *Compute axis-specific stationary noise measures*

Description

Compute axis-specific stationary noise measures.

Usage

```
static_axis_noise_metrics(x, cv_min_abs_mean_g = 0.005)
```

Arguments

`x`	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.
`cv_min_abs_mean_g`	Absolute mean-acceleration threshold in g below which a coefficient of variation is returned as NA. Default 0.005 g avoids unstable division for axes close to zero mean.

Value

One-row tibble with raw and linearly detrended sample SDs, first-five-minute raw SDs, CVs, ranges and maximum-/median-axis summaries. Acceleration fields use g unless their name explicitly specifies mg; CV fields are percentages.

Implementation

Implemented by `static_axis_noise_metrics` in `R/m_static.R`. See `implementation-guide` for the framework and function map.

Returned fields

For each prefix x, y, z and vm, the one-row output contains `n` (finite samples used), `mean_g`, `sd_g` (sample SD), `detrended_sd_g` (linear-residual SD), `sd_5min_g` (raw SD within the first five minutes, inclusive boundary), `min_g`, `max_g`, `range_g`, `cv_percent` and `detrended_cv_percent`. Each prefix is joined to the field name by an underscore. Linear-residual SD uses n-2; raw SD uses n-1.

`max_axis_sd_g` and `median_axis_sd_g` summarise raw X/Y/Z SDs; `max_axis_noise_sd_g` and `median_axis_noise_sd_g` summarise detrended X/Y/Z SDs. Their maximum-axis identifiers end in `_axis`; analogous `sd_5min` fields use the initial raw window. `static_n_samples` counts supplied rows.

`static_noise_floor_mg` is 1000 times maximum-axis detrended SD. The retained `static_noise_floor_5min_mg` field is instead 1000 times initial raw maximum-axis SD; it is not detrended. `static_raw_max_axis_sd_mg` is the full-window raw maximum-axis SD in mg. `vm_sd_mg`, `vm_noise_sd_mg` and `vm_sd_5min_mg` give VM raw, detrended and initial raw SDs in mg; `vm_noise_cv_percent` is detrended VM CV. These operational outcomes do not independently establish stationarity.

See Also

metronautica-package

`static_clip` *Select a stationary assessment interval*

Description

Select a stationary assessment interval.

Usage

```
static_clip(x, start_utc, duration_h)
```

Arguments

`x`	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.
`start_utc`	UTC window start; POSIXct or parseable text.
`duration_h`	Required positive window duration in hours; the end is exclusive. No duration is assumed. Requested bounds, actual sample span and boundary coverage are retained in processing$static_window metadata. Incomplete boundary coverage warns; this is not an internal-gap or stationarity test.

Value

The selected acceleration table; the end of the static interval is excluded.

Implementation

Implemented by `static_clip` in `R/m_static.R`. See implementation-guide for the framework and function map.

Container behaviour

Returns an acceleration table for both table and bundle input. A supplied parser bundle is unwrapped from `data`; a dynamic bundle is unwrapped from `full`. Metadata is attached to the selected table; no stage bundle is returned or updated. Each selected interval receives freshly computed QC.

See Also

metronautica-package

static_drift *Estimate signed drift from one-second means*

Description

Estimate signed drift from one-second means.

Usage

```
static_drift(d1)
```

Arguments

d1	One-second summary table returned by static_1hz_means, with elapsed_h and axis/VM mean columns. elapsed_h is time since the first populated UTC-second bin, in hours. This is a derived table, not a raw parser bundle.

Value

A list with `vm_drift_g_per_h`, `x_drift_g_per_h`, `y_drift_g_per_h` and `z_drift_g_per_h`: signed slopes in g/h. `fit_vm` is the fitted VM `lm` object. Fitted axis-model objects are not returned.

Method

Fit separate ordinary least-squares models of one-second VM and axis means on elapsed_h. Return the signed slopes in g/h and the fitted VM model. Selection of a stationary interval is the caller's responsibility; fitting a slope does not establish physical sensor drift in uncontrolled movement.

Implementation

Implemented by `static_drift` in `R/m_static.R`. See `implementation-guide` for the framework and function map.

See Also

`metronautica-package`

static_metrology_from_tbl *Characterise a stationary recording*

Description

Estimate stationary offset, linear drift, signal noise and non-overlapping Allan deviation.

Usage

```
static_metrology_from_tbl(x, instrument = NA_character_, device_id =
    NA_character_, device_vendor = NA_character_, device_model =
    NA_character_, fs_nominal_hz = NA_real_, fs_derived_hz =
    NA_real_, out_dir = NULL, start_utc, duration_h,
    title_prefix = NULL, verbose = FALSE, progress = TRUE)
```

Arguments

x	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.
instrument	Optional character label identifying the instrument in summaries and filenames; otherwise use vendor/model metadata when available.
device_id	Optional unit identifier for summaries and filenames; otherwise use acquisition$device_id metadata. This label does not select calibration coefficients.
device_vendor	Optional manufacturer label; otherwise use acquisition metadata. Does not alter decoding or calculations.
device_model	Optional device-model label; otherwise use acquisition metadata. Does not alter decoding or calculations.
fs_nominal_hz	Optional nominal recording frequency in Hz for summary identification; acquisition metadata supplies the fallback. Static calculations use their implemented timestamp-derived frequency, rather than this label.
fs_derived_hz	Optional timestamp-derived frequency label in Hz for the summary; metadata/timestamps provide a fallback. It does not override the implemented static calculation windows.
out_dir	NULL returns results in memory without writing files. Otherwise, create/use this output directory for the documented CSV, RDS and diagnostic plot files.
start_utc	UTC window start; POSIXct or parseable text.
duration_h	Required positive window duration in hours; the end is exclusive. No duration is assumed. Requested bounds, actual sample span and boundary coverage are retained in processing$static_window metadata. Incomplete boundary coverage warns; this is not an internal-gap or stationarity test.
title_prefix	Character prefix for diagnostic plot titles; does not change computed outcomes.
verbose	Logical; print processing messages. Does not change calculations.
progress	Logical; display a text progress bar. Does not change calculations.

Details

Offset is the mean one-second vector magnitude over `elapsed_h <= 5/60`, minus 1 g. Drift is the least-squares slope of one-second means against elapsed hours, in g/h. Static noise floor is the maximum axis-specific residual SD after linear detrending over the selected window, reported in mg. The first-five-minute maximum-axis SD is raw, not detrended. Raw SD uses $n - 1$; linearly detrended residual SD uses $n - 2$. The stationary coefficient of variation is a percentage with a near-zero mean guard. Allan deviation uses non-overlapping cluster means. The maximum-axis Allan minimum is derived as the maximum of the three individual axis minima over evaluated averaging times. Full definitions are in `measurement-definitions`. The input window must be stationary; the function does not establish that condition or infer a sensor calibration.

Value

An invisible list: `summary` (one-row table), `allan` (cluster durations and deviations), `drift_plot`, `allan_plot`, `clipped_data`, `one_hz` (axis/VM means and elapsed hours), `meta` and `files` (written paths, if requested).

Implementation

Implemented by `static_metrology_from_tbl` in `R/m_static.R`. See `implementation-guide` for the framework and function map.

Summary fields

The one-row `summary` includes instrument/unit identifiers, actual first-to-last sample span in `trial_duration_h`, four `*_drift_g_per_h` slopes, `vm_offset_g_5min`, nominal/derived rate labels in Hz, and observed `clip_start_utc/clip_end_utc`. Axis/VM noise columns follow `static_axis_noise_metrics`. Axis-specific Allan minima and their evaluated averaging times follow `static_allan_minima_wide`. Actual sample span differs from requested window duration by endpoint sampling and any incomplete coverage; consult `meta$processing$static_window`.

See Also

`metronautica-package`

---

static_noise_summary_from_tbl

Summarise stationary axis noise

Description

Summarise stationary axis noise.

Usage

```
static_noise_summary_from_tbl(x, instrument = NA_character_,
    device_id = NA_character_, device_vendor = NA_character_,
    device_model = NA_character_, fs_nominal_hz = NA_real_,
    fs_derived_hz = NA_real_, out_dir = NULL, start_utc, duration_h,
    verbose = FALSE, progress = TRUE)
```

Arguments

`x`	Acceleration data frame/tibble with POSIXct time and numeric ax_g, ay_g, az_g in g, or a parser/clip bundle containing data/full and optional meta. The data attribute takes precedence over bundle metadata. No samples are repaired by input conversion.
`instrument`	Optional character label identifying the instrument in summaries and filenames; otherwise use vendor/model metadata when available.
`device_id`	Optional unit identifier for summaries and filenames; otherwise use acquisition$device_id metadata. This label does not select calibration coefficients.
`device_vendor`	Optional manufacturer label; otherwise use acquisition metadata. Does not alter decoding or calculations.
`device_model`	Optional device-model label; otherwise use acquisition metadata. Does not alter decoding or calculations.
`fs_nominal_hz`	Optional nominal recording frequency in Hz for summary identification; acquisition metadata supplies the fallback. Static calculations use their implemented timestamp-derived frequency, rather than this label.

fs_derived_hz	Optional timestamp-derived frequency label in Hz for the summary; metadata/timestamps provide a fallback. It does not override the implemented static calculation windows.
out_dir	NULL returns results in memory without writing files. Otherwise, create/use this output directory for the documented CSV, RDS and diagnostic plot files.
start_utc	UTC window start; POSIXct or parseable text.
duration_h	Required positive window duration in hours; the end is exclusive. No duration is assumed. Requested bounds, actual sample span and boundary coverage are retained in processing$static_window metadata. Incomplete boundary coverage warns; this is not an internal-gap or stationarity test.
verbose	Logical; print processing messages. Does not change calculations.
progress	Logical; display a text progress bar. Does not change calculations.

Value

An invisible list containing summary (one-row noise/identifier table), clipped_data (selected acceleration table), meta (including static-window coverage) and files (written paths, or NULL). This is not itself a one-row table.

Implementation

Implemented by static_noise_summary_from_tbl in R/m_static.R. See implementation-guide for the framework and function map.

Summary fields

Noise and variability fields follow static_axis_noise_metrics; the report adds instrument/unit labels, nominal/derived frequency labels in Hz, actual first-to-last sample duration in hours and observed clip bounds. The selected data and boundary-coverage metadata are returned separately. This report does not calculate drift, initial VM offset or Allan curves; use static_metrology_from_tbl for those outcomes.

See Also

metronautica-package

---

static_offset_vm_5min *Estimate signed one-g vector-magnitude offset bias*

Description

Estimate signed one-g vector-magnitude offset bias.

Usage

```
static_offset_vm_5min(d1)
```

Arguments

d1	One-second summary table returned by static_1hz_means, with elapsed_h and axis/VM mean columns. elapsed_h is time since the first populated UTC-second bin, in hours. This is a derived table, not a raw parser bundle.

Value

Signed initial-window VM offset in g relative to 1 g.

Method

Select one-second bins satisfying elapsed_h <= 5/60, average vm_mean and subtract 1 g. The inclusive elapsed-bin rule is retained exactly. This measures VM offset relative to gravity, not an axis-specific multi-orientation calibration.

Implementation

Implemented by `static_offset_vm_5min` in `R/m_static.R`. See `implementation-guide` for the framework and function map.

See Also

`metronautica-package`

Index

accel_meta, 4
accel_metric_agcounts, *3*, 5, *31–33*, *47*
accel_metric_ai, 8, *32*
accel_metric_enmo, 9, *32*, *42*
accel_metric_mad, 10, *32*, *42*
accel_metric_mims, 11, *32*
accel_metric_odba, 13, *32*
accel_metric_rocam, 15, *32*
accel_metric_svmbp, 16, *32*, *42*
accel_metric_svmg, 17, *32*, *42*
accel_metric_vm, 18, *32*
accel_metric_z_angle, 19, *32*
accel_metric_z_angle_change, 20, *32*
allan_nonoverlap, 21, *46*
as_accel_tbl, 22, *27*
attach_qc, 23, *45, 46*

clip_bundle, 24, *27*
clip_dynamic_stages, 25, *44, 45*
common-data-format, 27
compute_accel_summaries, *4*, *7*, 27, *42*, *45, 46*, *49*
compute_ai_noise_var_from_static, 34

dm_apply_vm_calibration_bundle, *25*, 35
dm_apply_vm_calibration_tbl, 36, *46*
dm_fit_vm_calibration, 37, *46*
dynamic_metrology_report, *4*, 38, *45, 46*, *49*, *53*

feature-comparison, 42
filter_accel_tbl, 43

getting-started, 44

implementation-guide, 45
invalidate_qc, *46*, 48

m_dynamic_response, *46*, 49, *53*
m_load_sample_data, *4*, 51, *53*
measurement-definitions, 48
metronautica *(*metronautica-package*)*, 3
metronautica-package, 3

native-recording-examples, 52
native-study-examples *(*native-recording-examples*)*, 52

parse_raw_auto, *4*, *7*, *45, 46*, *52, 53*, 54
plot_accel_clip, 56
plot_processed_metric, 57
plot_static_allan, 57
plot_static_drift_vm, 58

qc_accel_tbl, *4*, *7*, *27*, *46*, 59

static_1hz_means, *46*, 60
static_allan, *46*, 61
static_allan_minima_wide, 62, *67*
static_axis_noise_metrics, *46*, 63, *67, 68*
static_clip, *45*, 64
static_drift, *46*, 65
static_metrology_from_tbl, *4*, *45*, *49*, *53*, 65, *68*
static_noise_summary_from_tbl, 67
static_offset_vm_5min, *46*, 68